arXiv:2604.22708v1 [cs.MA] 24 Apr 2026

Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems

Mengzhuo Chen[†,‡,§], **Junjie Wang**[†,‡,*], **Fangwen Mu**[†,‡,§], **Yawen Wang**[†,‡], **Zhe Liu**[†,‡], **Huanxiang Feng**[†,‡,§], **Qing Wang**[†,‡,*]

[†]State Key Laboratory of Complex System Modeling and Simulation Technology
[‡]Institute of Software, Chinese Academy of Sciences
[§]University of Chinese Academy of Sciences, Beijing, China

Abstract

Failure attribution, i.e., identifying the responsible agent and decisive step of a failure, is particularly challenging in LLM-based multi-agent systems (MAS) due to their natural-language reasoning, nondeterministic outputs, and intricate interaction dynamics. A reliable benchmark is therefore essential to guide and evaluate attribution techniques. Yet existing benchmarks rely on partially observable traces that capture only agent outputs, omitting the inputs and context that developers actually use when debugging. We argue that failure attribution should be studied under full execution observability, aligning with real-world developer-facing scenarios where complete traces, rather than only outputs, are accessible for diagnosis. To this end, we introduce TraceElephant, a benchmark designed for failure attribution with full execution traces and reproducible environments. We then systematically evaluate failure attribution techniques across various configurations. Specifically, full traces improve attribution accuracy by up to 76% over a partial-observation counterpart, confirming that missing inputs obscure many failure causes. TraceElephant provides a foundation for follow-up failure attribution research, promoting evaluation practices that reflect real-world debugging and supporting the development of more transparent MASs.

1 Introduction

If there is one constant in the evolution of software, it is the persistent occurrence of failures (Charette, 2005). When it does, the critical first step is to assign responsibility for the failure to a specific component, i.e., failure attribution or localization, which enables developers to focus debugging efforts, guide the design of patches or architectural improvements. Traditional techniques for this task, ranging from statistical debugging (Zheng et al., 2006) and delta debugging (Misherghi and Su, 2006) to more recent learning-based approaches (Wong et al., 2016; Zou et al., 2019), operate under the assumption that system states are discrete, executions are traceable, and component behaviors are largely deterministic.

The rise of LLM-based multi-agent systems (MASs) fundamentally challenges these assumptions and complicates the attribution problem. In these systems, complex tasks are decomposed and coordinated across multiple agents whose primary reasoning and communication medium is natural language (Guo et al., 2024; Li et al., 2024). This introduces two layers of complexity for fault attribution. First, the problem-solving process in MASs often involves complex interactions among multiple LLM-powered agents, between agents and external tools, and within the internal reasoning processes of the LLMs themselves. These interactions complicate system logs, challenging the interpretation of system behavior and hindering rapid root cause identification. Second, the system actions and their resulting states are recorded in natural language within the log. The inherent ambiguity of natural language further impedes the precise characterization of operations and states.

To address the challenge of fault attribution in MAS, several technical approaches have been proposed, such as ECHO (Banerjee et al., 2025), Agen-Tracer (Zhang et al., 2025a), GraphTracer (Zhang et al., 2025b), and FAMAS (Ge et al., 2025). On another front, to effectively evaluate how these techniques perform in real-world MAS settings, the foundation lies in having benchmarks that accurately reflect authentic fault attribution scenarios.

To the best of our knowledge, Who&When (Zhang et al., 2025c) is currently the only benchmark specifically designed for failure attribution in LLM-based MAS. It provides partially observable execution traces that include only the agents' outputs, without the corresponding inputs such as

*Corresponding authors

arXiv:2604.22708v1 [cs.MA] 24 Apr 2026

洞见全貌：基于大语言模型的多智能体系统中的故障归因基准测试

陈孟卓[†,‡,§], 王俊杰[†,‡,*], 穆方文[†,‡,§], 王雅雯[†,‡], 刘哲[†,‡], 冯欢翔[†,‡,§], 王青[†,‡,*]

[†]复杂系统建模与仿真技术国家重点实验室
[‡]中国科学院软件研究所[§]中国科学院大学，北京，中国

摘要

故障归因，即识别导致故障的责任智能体和决定性步骤，在基于大语言模型的多智能体系统（MAS）中极具挑战性，原因在于其涉及自然语言推理、非确定性输出以及复杂的交互动力学。因此，可靠的基准测试对于指导与评估归因技术至关重要。然而，现有的基准测试依赖部分可观测轨迹，仅捕获智能体输出，而遗漏了开发者在调试过程中实际使用的输入和上下文。我们认为，故障归因应当在完全可观测执行轨迹的条件下进行研究，以符合真实世界中开发者面临的场景，在这些场景中，完整的执行轨迹（而不仅仅是输出）可用于故障诊断。为此，我们提出了TraceElephant，这是一个针对具备完整执行轨迹和可复现环境的故障归因而设计的基准测试。随后，我们系统地评估了各种配置下的故障归因技术。具体而言，完整轨迹相较于仅基于部分观测的基准测试，使归因准确率最高提升了76%，证实了缺失的输入会掩盖许多故障原因。TraceElephant为后续的故障归因研究奠定了基础，推动反映真实世界调试工作的评估实践，并支持更透明的多智能体系统的开发。

1 引言

如果说在软件的演进中存在一个不变的常量，那就是故障的持续发生（Charette，2005）。当故障发生时，关键的第一步是将故障的责任归于某个特定组件，即故障归因或定位，这使得开发者能够将调试努力集中于特定方向，并指导补丁或架构改进的设计。用于这一任务的传统技术，从统计调试（Zheng 等人，2006）和差异调试（Misherghi 和 Su，2006）到更近期的基于学习的方法（Wong 等人，2016；Zou 等人，2019），都是在系统状态离散、执行过程可追踪以及组件行为在很大程度上具有确定性的假设下运行的。

基于大语言模型的多智能体系统的兴起从根本上挑战了这些假设，并使归因问题更加复杂。在这些系统中，复杂任务被分解并通过多个智能体进行协调，其主要推理和通信媒介是自然语言（Guo et al., 2024; Li et al., 2024）。这为故障归因引入了两个层面的复杂性。首先，多智能体系统中的问题求解过程通常涉及多个大语言模型驱动的智能体之间、智能体与外部工具之间，以及大语言模型自身内部推理过程中的复杂交互。这些交互使系统日志变得复杂，挑战了对系统行为的解释，并阻碍了快速的根本原因识别。其次，系统操作及其产生的状态以自然语言的形式记录在日志中。自然语言的固有歧义进一步阻碍了对操作和状态的精确描述。

为应对多智能体系统中的故障归因挑战，研究者们提出了多种技术方案，例如ECHO（Banerjee et al., 2025）、Agen- Tracer（Zhang et al., 2025a）、GraphTracer（Zhang et al., 2025b）以及FAMAS（Ge et al., 2025）。另一方面，为了有效评估这些技术在真实世界多智能体系统设置中的表现，其基础在于拥有能够准确反映真实故障归因场景的基准测试。

据我们所知，Who&When（Zhang等人，2025年）是目前唯一专为基于大型语言模型的多智能体系统的故障归因而设计的基准。它提供部分可观察的执行轨迹，这些轨迹仅包含智能体的输出，而不包含相应的输入，例如

*通信作者

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

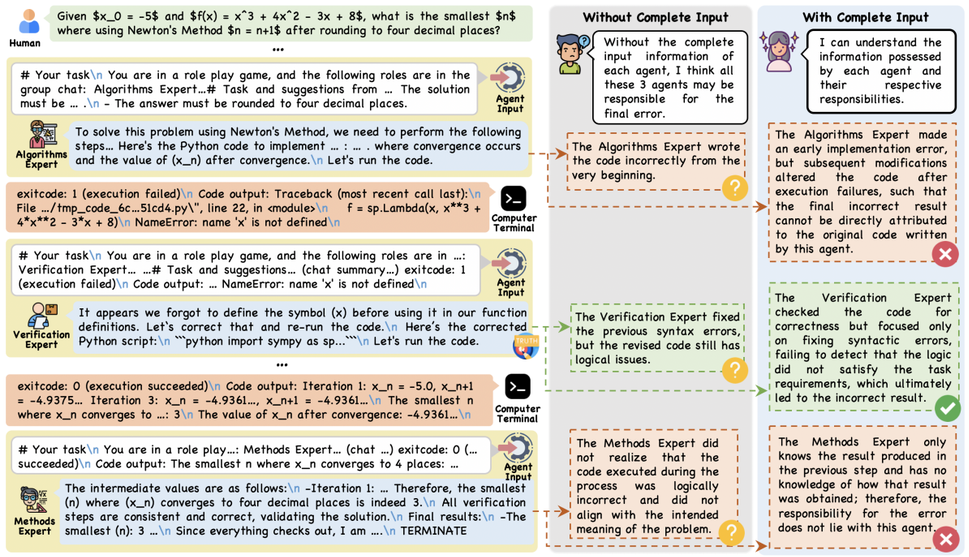


Figure 1: A failure case (from Who&When benchmark) illustrating the limitation of partial observability. When only agent outputs are visible and critical inputs are absent, localizing the decisive failure step becomes difficult.

original task instructions, prompts, or contextual messages. This restricted setting can be suitable for evaluating attribution methods under certain black-box scenarios, where internal inputs or intermediate states are inaccessible (e.g., debugging deployed agents on external platforms).

However, in many practical developer-facing debugging scenarios, developers typically have access to substantially richer execution information, including task instructions, prompts, intermediate messages, tool invocations, and environment states. More importantly, we view failure attribution as a developer-centric task conducted in the context of system debugging and iterative refinement, with the explicit goal of producing actionable insights for repairing and improving MASs. Achieving this goal critically depends on access to the inputs, intermediate states, and environment interactions that drive agent behavior. In contrast, black-box settings, where only outputs are observable, leave many failures ambiguous and thus offer limited practical value for guiding system fixes. For instance, our analysis of the 184 failure cases in Who&When benchmark shows that in at least 21% of instances, developers cannot reliably perform failure attribution using output-only logs. We illustrate this in Figure 1 with an adapted case from Who&When, where we restored the missing inputs by re-running the original system. Under partial observability, the failure cause remains ambiguous, whereas with full inputs, the ambiguity is largely resolved, enabling precise step-level localization.

To fill this gap, we construct TraceElephant, the first benchmark specifically designed for developer-facing failure attribution in LLM-based MASs by "seeing the whole elephant", i.e., providing access to the complete execution narrative of an MAS. Although we use MAS as the main terminology, the attribution unit in TraceElephant is a functional component rather than necessarily a separate agent entity, so the benchmark also applies to single-agent scaffold systems whose planning, orchestration, and tool-use modules make distinct decisions. Compared with existing benchmarks in this field (i.e., Who&When), TraceElephant differs in two key aspects: (1) TraceElephant collects step-by-step execution traces from multiple representative agentic systems, where each trace records agent-level actions, natural language inputs and outputs, tool and environment interactions, agent configurations and system architecture, and (2) it accompanies each trace with a reproducible execution environment, enabling controlled re-execution, state inspection, and interactive hypothetical debugging queries (e.g., "what if this agent had received different input?").

Building upon this benchmark, we evaluate the automated failure attribution techniques under

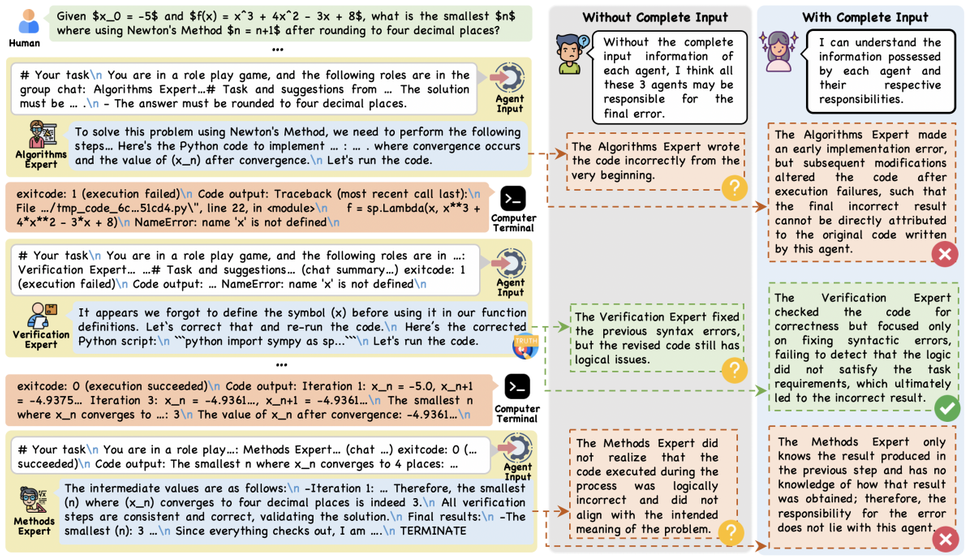


图1：Who&When基准中的一个故障案例，说明了部分可观察性的局限性。当仅可见智能体输出而关键输入缺失时，难以定位决定性故障步骤。

原始任务指令、提示词或上下文消息。这种受限场景适用于在特定黑盒场景下评估归因方法，其中内部输入或中间状态不可访问（例如，在外部平台上调试已部署的智能体）。

然而，在许多面向开发者的实际系统调试场景中，开发者通常能够获取到丰富得多的执行信息，包括任务指令、提示词、中间消息、工具调用以及环境状态。更重要的是，我们将故障归因视为一项在系统调试与迭代优化背景下进行的、以开发者为中心的任务，其明确目标是为修复和改进多智能体系统（MASs）生成可操作的见解。实现这一目标，关键在于能够访问驱动智能体行为的输入、中间状态和环境交互。相比之下，在黑盒设置下，仅能观察到输出，这使得许多故障原因存在歧义，因此在指导系统修复方面的实用价值有限。例如，我们对 Who&When基准 中184个故障案例的分析显示，在至少21%的情况下，开发者无法仅使用仅含输出的日志可靠地进行故障归因。我们在图1中通过一个改编自 Who&When基准 的案例对此进行了说明，在该案例中，我们重新运行原始系统以恢复缺失的输入。在部分可观察性下，故障原因仍然存在歧义；而在使用完整输入时，歧义在很大程度上得以解决，从而能够实现精确的步骤级定位。

为了填补这一空白，我们构建了TraceElephant，这是首个专为"看见完整的大象"而设计的基准测试，旨在实现面向开发者的大型语言模型（LLM）多智能体系统（MAS）故障归因，即提供对MAS完整执行叙述的访问。尽管我们主要使用MAS作为术语，但TraceElephant中的归因单元是功能组件，而非必然是独立的智能体实体。因此，该基准测试也适用于单体智能体脚手架系统，其中规划、编排和工具使用模块会做出不同的决策。与现有该领域的基准测试（即Who&When）相比，TraceElephant在两个关键方面有所不同：（1）TraceElephant从多个代表性智能体系统中收集逐步执行轨迹，每条轨迹记录了智能体级操作、自然语言输入和输出、工具与环境交互、智能体配置以及系统架构；（2）它为每条轨迹配备了一个可复现的执行环境，从而支持受控重新执行、状态检查以及交互式假设性调试查询（例如，"如果此智能体收到不同的输入，会发生什么？"）。

在此基准测试的基础上，我们评估自动化故障归因技术在

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

various configurations. The experimental results demonstrate that with complete trace information, it achieves an average attribution accuracy of 65.9% at the agent level and 30.3% at the step level. This represents an improvement of 22% in agent-level accuracy and 76% in step-level accuracy over the performance on output-only traces (i.e., similar to Who&When benchmark), underscoring the critical role of full observability. With the running environment, the step-level accuracy can further improve by 10%. Our analysis further reveals that step-level attribution is more sensitive to missing information than agent-level attribution, highlighting the finer-grained, context-dependent nature of localizing the precise failure step. Moreover, performance can also vary across different MAS architectures, agent types, and step positions, indicating the need for architecture-aware attribution approaches. We also provide actionable implications and takeaways for developing more effective attribution techniques and designing more debuggable MASs.

On one hand, this work highlights the necessity of full trace information for reliable fault attribution, indicating that developers should, whenever possible, incorporate all accessible execution details when performing this task. On the other hand, it also suggests that the field would benefit from more diverse benchmarks to evaluate attribution techniques from multiple perspectives, thereby contributing to a more solid and cumulative understanding of failure attribution in MAS.

The contributions of this work are as follows.

- We are the first to study failure attribution of LLM-based MASs from a developer-facing scene, where full execution traces and reproducible execution environment are available.
- We develop TraceElephant[1], a failure attribution benchmark that instantiates the above paradigm by providing a collection of annotated execution traces. It consists of 220 failure traces collected from three representative agentic systems, including both multi-agent orchestration and a single-agent tool-centric scaffold, with each annotated with the responsible component and decisive failure step.
- We conduct extensive experiments on TraceElephant, examining how failure attribution behaves under various configurations, and providing related implications.

[1]https://github.com/TraceElephant/TraceElephant

2 Problem Definition

We consider an LLM-based multi-agent system (MAS) composed of a finite set of functional components $\mathcal{A} = \{a_1, \ldots, a_N\}$ collaboratively performing a task τ. These components may correspond to explicit agents in a classical MAS, or to functional modules in a single-agent scaffold, such as planning, orchestration, and tool-use components. The system executes in discrete steps under a turn-based protocol, where exactly one component is selected to act at each step. At step t, the acting component $a(t)$ receives input x_t and produces output y_t. The execution trace at step t is recorded as

$$o_t = \big(x_t,\, y_t,\, a(t),\, \textit{step_id}_t,\, \textit{agent_id}_t\big).$$

The task outcome is determined by the complete execution trace. In case of failure, we aim to attribute responsibility at both the step and agent level: step-level attribution identifies the earliest point at which failure becomes inevitable, while agent-level attribution identifies the component responsible for the failure at that step. This definition follows a role-aware and recoverability-aware principle rather than a purely chronological notion of the first visible mistake. If an upstream mistake remains recoverable because a later verifier or orchestrator is explicitly responsible for checking and correcting it, the decisive failure step is attributed to the point where that recovery opportunity is missed. For example, if an agent produces a hallucinated fact at step t but a verifier at step $t+2$ is expected to detect such errors and fails to do so, the failure is attributed to the verifier step, because the system remains recoverable until that responsibility is not fulfilled. For the task of failure attribution in MAS, the goal is to determine both the step-level and agent-level responsibility given the full failure trace.

For a full formalization, including definitions, notation, and equations, refer to Appendix A.1.

3 Benchmark Construction

We construct TraceElephant, a benchmark for failure attribution in LLM-based MAS. Each instance

多种配置。实验结果表明，在具备完整轨迹信息的情况下，该方法在智能体级和步骤级的平均归因准确率分别达到65.9%和30.3%。与仅输出轨迹（即类似于Who&When基准）的性能相比，智能体级准确率提升了22%，步骤级准确率提升了76%，凸显了完全可观测性的关键作用。结合运行环境，步骤级准确率还可进一步提升10%。我们的分析进一步揭示，步骤级归因比智能体级归因对缺失信息更为敏感，这凸显了精确定位故障步骤所具有的更细粒度、依赖上下文的特性。此外，性能在不同多智能体系统架构、智能体类型和步骤位置上也存在差异，表明需要采用架构感知归因方法。我们还提供了对开发更有效的归因技术和设计更可调试多智能体系统的可行启示和经验总结。

一方面，本工作凸显了完整轨迹信息对于可靠故障归因的必要性，表明开发者在执行该任务时，应尽可能纳入所有可获取的执行细节。另一方面，它同样建议，该领域需要从多个角度评估归因技术，从而受益于更多样化的基准测试，进而有助于建立对多智能体系统中故障归因更为坚实和累积性的理解。

本研究的贡献如下。

•我们是首个从开发者视角研究基于大语言模型的多智能体系统故障归因的工作，在该场景中，完整的执行轨迹和可复现的执行环境均可获得。

•我们开发了TraceElephant1，这是一个故障归因基准测试，通过提供一组标注的执行轨迹来实例化上述范式。它包含220条故障轨迹，这些轨迹收集自三个具有代表性的智能体系统，涵盖多智能体编排和单智能体工具中心脚手架，每条轨迹都标注了责任组件和决定性故障步骤。

•我们在TraceElephant上进行了广泛的实验，考察故障归因在各种配置下的表现，并提供相关启示。

[1]https://github.com/TraceElephant/TraceElephant

2 问题定义

我们考虑一个由有限数量的功能组件 $\mathcal{A} = \{a_1, \ldots, a_N\}$ 组成的基于大型语言模型（LLM）的多智能体系统（MAS），这些组件协同执行任务 τ。这些组件可能对应于经典多智能体系统中的显式智能体，也可能对应于单智能体脚手架中的功能模块，例如规划、编排和工具使用组件。该系统在基于回合的协议下以离散步骤执行，在每一步中恰好选择一个组件采取行动。在步骤 t，行动组件 $a(t)$ 接收输入 x_t 并产生输出 y_t。步骤 t 的执行轨迹记录为

$$o_t = \big(x_t,\, y_t,\, a(t),\, \textit{step_id}_t,\, \textit{agent_id}_t\big).$$

任务结果由完整执行轨迹决定。在发生失败的情况下，我们旨在对步骤和智能体进行归因：步骤级归因识别出失败变得不可避免的最早点，而智能体级归因识别出在该步骤中对失败负责的智能体。该定义遵循角色感知和可恢复性感知原则，而非纯粹基于首个可见错误的时间顺序概念。如果一个上游错误仍然可恢复，因为后续的验证器或编排器明确负责检查和纠正该错误，那么决定性故障步骤将归因于错过该恢复机会的点。例如，如果一个智能体在步骤 t 生成了一个幻觉事实，但步骤 $t+2$ 的验证器本应检测到此类错误却未能做到，那么失败将归因于验证器步骤，因为系统在该责任未被履行之前保持可恢复状态。对于多智能体系统中的故障归因任务，目标是在给定完整故障轨迹的情况下，确定步骤级和智能体级的责任。

有关完整的形式化内容，包括定义、符号和方程，请参阅附录 A.1。

3 基准构建

我们构建了TraceElephant，一个用于基于大语言模型的多智能体系统中故障归因的基准测试。每个实例

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

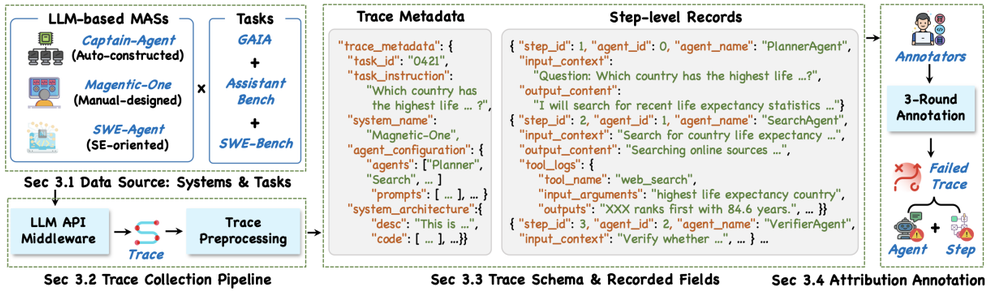


Figure 2: Overview of TraceElephant.

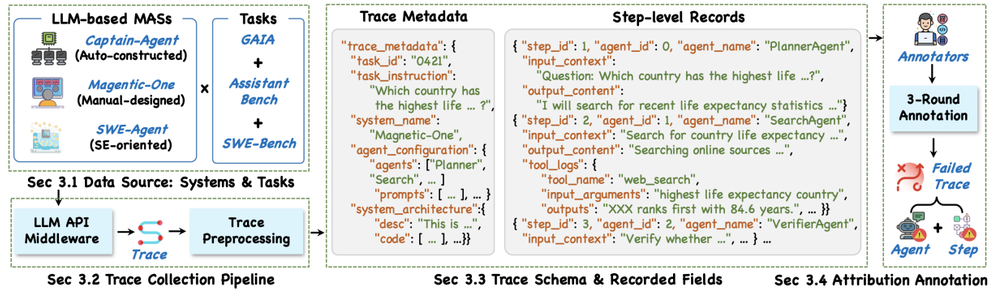


图2：TraceElephant 概览。

is grounded in an executable MAS and is associated with a fully observable execution trace, as well as the annotated failure-responsible agent and decisive failure step.

3.1 Data Sources: Systems and Tasks

TraceElephant collects execution traces from three representative agentic systems across various tasks, aligning task design with each system's intended capabilities, as demonstrated in Table 1. These systems cover dynamically assembled agent teams, fixed-role multi-agent orchestration, and a single-agent tool-centric scaffold, allowing the same attribution formulation to apply across both explicit multi-agent interactions and scaffolded single-agent workflows. Detailed descriptions of the systems, task sources, run configurations are provided in Appendix A.2.

System	Task Source	# Traces	# Failed
Captain-Agent	GAIA	126	73
Captain-Agent	AssistantBench	21	12
Magentic-One	GAIA	119	74
Magentic-One	AssistantBench	30	17
SWE-Agent	SWE-Bench	84	44
Total	All	380	220

Table 1: Overview of execution traces in TraceElephant.

3.2 Trace Collection Pipeline

TraceElephant adopts an automated collection pipeline designed to capture complete execution traces while preserving the original structure of agent interactions. At its core, a lightweight LLM API middleware transparently intercepts and captures all LLM requests, responses, and subsequent tool interactions without modifying the original agent implementations. The raw traces then undergo targeted pre-processing, extracting only basic attributes such as agent names and step types, to maintain maximal fidelity to the original execution flow. More details are provided in Appendix A.3.

3.3 Trace Schema and Recorded Fields

Each instance in TraceElephant benchmark consists of a complete execution trace generated by running a multi-agent system on a given task, as well as the executable system associated with the runnable code and configurations for this specific trace instance.

(1) Trace metadata. Each execution trace is associated with a set of trace-level metadata that describes the execution context as a whole. These metadata fields include: (1) *task_id* and *task_instruction*, indicating the task being solved; (2) *system_name*, identifying the multi-agent system that produced the trace; (3) *agent_configuration*, the runtime setup defining the agent roster, prompts, and toolset; and (4) *system_architecture*, the design documentation and implementation code defining the system's structural blueprint.

(2) Step-level records. An execution trace is composed of an ordered sequence of steps, where each step corresponds to a single agent action. For each step, TraceElephant records a set of observable fields, organized into input and output fields, that are directly exposed by the running system.

(i) Step input fields. They capture all information provided to an agent when an action is executed. Specifically, the recorded input fields include: (1) *step_id*, a unique identifier indicating the global execution order of the step; (2) *agent_id* and *agent_name*, identifying the agent responsible for the action; (3) *input_context*, which stores the complete input information supplied to the agent. This field typically includes the original task instruction, intermediate messages exchanged among agents, and any system-constructed contextual information available at execution time.

都基于一个可执行的多智能体系统，并关联有一条完全可观察的执行轨迹，以及经过标注的故障责任代理和决定性故障步骤。

3.1 数据来源：系统与任务

TraceElephant 从三个具有代表性的智能体系统中收集执行轨迹，涵盖多种任务，并使任务设计与各系统的预期能力相匹配，如表1所示。这些系统涵盖了动态组装的智能体团队、固定角色的多智能体编排以及单智能体工具中心支架，使得同一归因表述能够适用于显式多智能体交互和支架式单智能体工作流。关于系统、任务来源和运行配置的详细描述见附录 A.2。

系统	任务来源	# 执行轨迹	# 失败
Captain-Agent	GAIA	126	73
Captain-Agent	AssistantBench	21	12
Magentic-One	GAIA	119	74
Magentic-One	AssistantBench	30	17
SWE-Agent	SWE-Bench	84	44
总计	All	380	220

表1：TraceElephant 中执行轨迹的概述。

3.2 轨迹收集流水线

TraceElephant 采用自动化收集流水线，旨在捕获完整执行轨迹，同时保留智能体交互的原始结构。其核心是一个轻量级的 LLM API 中间件，它在不修改原始智能体实现的情况下，透明地拦截并捕获所有 LLM 请求、响应以及后续的工具交互。随后，原始轨迹经过有针对性的预处理，仅提取诸如智能体名称和步骤类型等基本属性，以最大限度地保持与原始执行流程一致。更多细节请参阅附录 A.3。

3.3 轨迹模式和记录字段

TraceElephant 基准测试中的每个实例包含通过在多智能体系统上运行特定任务而生成的完整执行轨迹，以及与该特定轨迹实例相关的可运行代码和配置所关联的可执行系统。

（1）轨迹元数据。每个执行轨迹都关联一组轨迹级元数据，用于整体描述执行上下文。这些元数据字段包括：（1）任务 _ 标识符和任务 _ 指令，指示正在解决的任务；（2）系统 _ 名称，用于识别生成该轨迹的多智能体系统；（3）智能体 _ 配置，即定义智能体名册、提示词和工具集的运行时配置；以及（4）系统 _ 架构，即定义系统结构蓝图的设计文档和实现代码。

（2）步骤级记录。执行轨迹由有序步骤序列构成，其中每个步骤对应一个智能体动作。对于每个步骤，TraceElephant 记录一组由运行系统直接暴露的可观察字段，这些字段组织为输入字段和输出字段。

(i) 步骤输入字段。它们捕获当动作被执行时提供给智能体的所有信息。具体而言，记录的输入字段包括：(1) step_id，一个表示步骤全局执行顺序的唯一标识符；(2) agent_id 和 agent_name，用于识别负责该动作的智能体；(3) input_context，用于存储提供给智能体的完整输入信息。该字段通常包括原始任务指令、智能体之间交换的中间消息，以及在执行时可用的系统构建的上下文信息。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

(ii) Step output fields. They record the observable results produced by the agent at the current step. These fields include: (1) *output_content*, which stores the response generated by the agent. (2) *tool_logs*, which (when available) records raw interaction logs with external tools or environments when such interactions occur. Each tool log typically includes the tool name, input arguments, outputs, and execution status.

All execution traces are stored in a structured JSON format. Each trace is represented as a JSON object containing trace-level metadata and an ordered list of step records, within which input and output fields are stored as nested objects, preserving all content in its raw, unprocessed form. We provide an example trace in Appendix A.4.

3.4 Failure Attribution Annotation

The annotation aims at acquiring failure attribution labels, i.e., (1) the component primarily responsible for the failure, and (2) the execution step where the failure originates. These labels are obtained through a multi-round expert annotation process designed to ensure reliability. On the first-round independent annotations, Krippendorff's alpha is 0.72 for agent-level labels and 0.64 for step-level labels, indicating substantial agreement for agent attribution and moderate-to-substantial agreement for step identification. Afterward, uncertain cases are jointly reviewed and finalized through expert consensus. Detailed annotation protocols are provided in Appendix A.5.

4 Failure Attribution Evaluation

4.1 Evaluation Design

Observability Configurations. We evaluate failure attribution performance under two complementary observability configurations reflecting practical debugging workflows. (1) ***Static***: Attribution is performed using the complete execution trace (including metadata, inputs, and outputs fields as shown in Section 3.3). (2) ***Dynamic***: In addition to the static trace, a replayable execution environment is provided, enabling controlled re-execution and counterfactual probing to verify or refine candidate attributions. We additionally utilize several variants of static configuration to further evaluate the performance under different levels of observability.

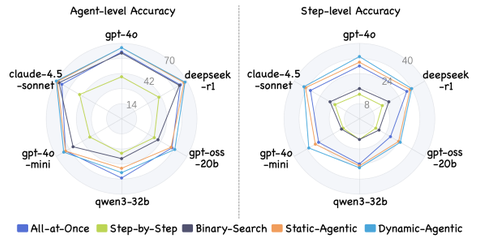


Figure 3: Comparison under different backbone LLMs.

Attribution techniques. We utilize five commonly-used techniques. There are three LLM-based prompting techniques, i.e., ***All-at-Once, Binary Search, Step-by-Step***, which differ in how the trace is provided to the LLM. There are two agent-based techniques. ***Static Agentic*** adopts mini-SWE-agent[2], which can navigate the trace information, retrieve related fields as needed, and make a conclusion gradually. ***Dynamic Agentic*** first proposes candidate failure attributions derived based on static agentic technique, with the candidate steps and agents. The method then re-runs the system from the corresponding execution point and issues counterfactual checks. Default setting of the following evaluations is Static Agentic.

Application Scenarios. Following existing studies (Zhang et al., 2025c), experiments are conducted both ***With ground truth*** (use this, if not explicitly stated) and ***Without ground truth*** scenarios, simulating different debugging contexts.

Evaluation Metrics. Predictions are evaluated for both responsible agent and decisive step, and we use ***agent-level accuracy*** and ***step-level accuracy***, following existing studies (Zhang et al., 2025c,b).

More details about the evaluation design are provided in Appendix A.6.

4.2 Results and Analysis

4.2.1 Performance Across Configurations

(1) Static vs. Dynamic Attribution Performance. Generally speaking, dynamic configuration can achieve the highest performance, especially for step-level accuracy, and within static configurations, the agentic technique is the best in most experimental settings.

As demonstrated in Table 2, with ground truth, failure attribution performance can reach an average of 33.3% and 30.3% step-level accuracy respectively in dynamic and static configuration. For

[2]2.4k stars as of January 2026. https://github.com/SWE-agent/mini-swe-agent/.

(ii) 步骤输出字段。它们记录智能体在当前步骤产生的可观察结果。这些字段包括：(1) output_content，用于存储由智能体生成的响应。(2) tool_logs，在可用时记录与外部工具或环境的原始交互日志（当此类交互发生时）。每条工具日志通常包括工具名称、输入参数、输出以及执行状态。

所有执行轨迹均以结构化 JSON 格式存储。每个执行轨迹表示为一个 JSON 对象，包含轨迹级元数据以及一个有序的步骤记录列表。其中，输入和输出字段以嵌套对象的形式存储，保留所有内容的原始未处理状态。我们在附录 A.4 中提供了一个示例执行轨迹。

3.4 故障归因标注

该标注旨在获取故障归因标签，即：(1) 对故障负主要责任的组件，以及 (2) 故障起源的执行步骤。这些标签通过旨在确保可靠性的多轮专家标注流程获得。在首轮独立标注中，智能体级标签的克里彭多夫阿尔法系数为 0.72，步骤级标签为 0.64，表明智能体归因具有实质性一致性，而步骤识别具有中等至实质性一致性。随后，不确定的案例将通过专家共识进行联合审查并最终确定。详细的标注协议见附录 A.5。

4 故障归因评估

4.1 评估设计

可观测性配置。我们在两种反映实际调试工作流的互补可观测性配置下评估失败归因性能。（1）静态配置：利用完整执行轨迹（包括元数据、输入和输出字段，如第3.3节所示）进行归因。（2）动态：除静态轨迹外，还提供可重放执行环境，支持受控重新执行和反事实探测，以验证或细化候选归因。此外，我们还利用多种静态配置变体，以进一步评估不同可观测性级别下的性能。

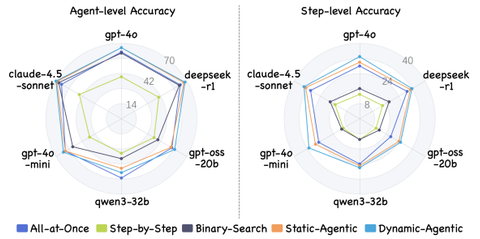


图3：不同主干大语言模型下的对比。

归因技术。我们采用了五种常用技术。其中包括三种基于大型语言模型的提示技术，即全量输入、二分查找和逐步输入，它们之间的区别在于向大型语言模型提供轨迹信息的方式不同。另外还有两种基于代理的技术。静态智能体采用mini-SWE智能体[2]，，该智能体可以导航轨迹信息，按需检索相关字段，并逐步得出结论。动态智能体首先基于静态智能体技术提出候选故障归因，包含候选步骤和智能体。随后，该方法从相应的执行点重新运行系统，并进行反事实检查。以下评估的默认设置为静态智能体。

应用场景。遵循现有研究（Zhang等人，2025c），实验在包含真值（除非另有说明，否则使用此设定）和不含真值两种场景下进行，以模拟不同的调试环境。

评估指标。预测结果针对责任代理和决定性步骤进行评估，并采用代理级准确率和步骤级准确率，遵循现有研究（Zhang等人, 2025c,b）。

有关评估设计的更多详情见附录A.6。

4.2 结果与分析

4.2.1 不同配置下的性能

(1) 静态与动态归因性能。一般而言，动态配置能够实现最高的性能，尤其是在步骤级准确率方面；而在静态配置中，智能体技术在大多数实验设置中表现最佳。

如表2所示，在有真值的情况下，故障归因性能在动态配置和静态配置下的步骤级准确率平均分别可达33.3%和30.3%。

[2]2.4k stars as of 2026年1月. https://github.com/SWE-agent/mini-swe-agent/.

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

System	Ground Truth	Static Configurations								Dynamic Config.	
		All-at-Once		Binary Search		Step-by-Step		Static Agentic		Dynamic Agentic	
		Agent	Step	Agent	Step	Agent	Step	Agent	Step	Agent	Step
Captain-Agent	**w/** Ground Truth	64.7	29.4	25.9	14.1	57.7	22.4	67.1	30.6	**68.2**	**32.9**
	w/o Ground Truth	**63.5**	22.4	24.7	9.4	44.7	17.7	58.8	24.7	61.2	**25.9**
Magentic-One	**w/** Ground Truth	58.2	25.2	38.5	13.2	63.7	20.9	67.0	30.8	**68.1**	**33.0**
	w/o Ground Truth	56.0	23.1	37.4	8.8	60.4	15.4	61.5	26.4	**61.5**	**27.5**
SWE-Agent	**w/** Ground Truth	**63.6**	29.6	52.3	11.4	61.4	6.8	**63.6**	29.6	**63.6**	**34.1**
	w/o Ground Truth	54.6	22.7	50.0	9.1	56.8	4.6	56.8	27.3	**59.1**	**29.6**
All-avg	**w/** Ground Truth	62.2	28.1	38.9	12.9	60.9	16.7	65.9	30.3	**66.7**	**33.3**
	w/o Ground Truth	58.0	22.7	37.4	9.1	54.0	12.5	59.1	26.1	**60.6**	**27.6**

Table 2: Performance (i.e., Agent-level accuracy and Step-level accuracy) comparison of failure attribution techniques across different agent systems under *with or without ground truth* scenario. We use bold to indicate the highest value, while underline indicating the second highest value.

Observability Configurations	Agent	Step
All-at-Once	**0.62**	**0.28**
w/o metadata	0.55	0.21
w/o input	0.54	0.18
w/o metadata & input	0.51	0.16
Static Agentic	**0.66**	**0.30**
w/o metadata	0.57	0.23
w/o input	0.56	0.19
w/o metadata & input	0.54	0.17

Table 3: Ablation study on observability configurations.

agent-level accuracy, these figures are 66.7% and 65.9% respectively. The dynamic method improves step-level attribution by 10% due to actively verifying candidate failure steps through controlled re-execution and counterfactual probing. This process helps filter out spurious candidates identified from static traces, thereby refining the attribution. Agent-level accuracy sees limited gains because it depends more on understanding agent roles and coordination logic, i.e., information already largely available in complete static traces.

Among all static configurations, agentic technique achieves the highest performance in almost all cases, which is likely because it allows to better trace responsibility through the chain of analysis and tool use that characterizes MAS failures. Furthermore, among other static configuration, All-at-Once performs relatively better than others. We conduct further analysis in Section 4.2.3.

(2) Ablation Study. Table 3 presents the ablation results for two representative techniques, while others show similar trend and are omitted for brevity. Full observability is essential for accurate failure attribution, and the absence of either metadata or input fields leads to noticeable performance degradation. Besides, step-level attribution is more sensitive to missing information than agent-level attribution, i.e., 76% vs. 22% accuracy drop, underscoring the finer-grained and more context-dependent nature of identifying the failure step.

(3) Effect of Backbone LLMs. Figure 3 demonstrates the performance in terms of different backbone LLMs. Broadly speaking, Claude-4.5-Sonnet, DeepSeek-R1, and GPT-4o exhibit relatively strong performance in both agent-level and step-level accuracy, likely due to their advanced reasoning architectures and strong contextual understanding. In contrast, Qwen3-32B and GPT-OSS-20B show weaker performance, which may be attributed to their smaller parameter scales and consequently limited capacity for sustained multi-step reasoning and fine-grained causal tracing.

(4) With vs. Without Ground Truth. Performance consistently declines across all systems and techniques when ground truth is unavailable. This confirms that access to a reference outcome (e.g., a correct answer or test pass/fail signal) provides crucial guidance for attribution, especially for finer-grained step attribution. Agentic methods (especially Dynamic Agentic) show relatively smaller performance degradation without ground truth. This suggests their interactive validation (replay and counterfactual checks) can partially compensate for the missing reference signal by testing behavioral hypotheses.

4.2.2 Failure & Attribution Patterns

(1) Failure Responsible Agents. Figure 4 illustrates the distribution of failure-prone agent types

系统	真值	静态配置						动态配置	
		全量输入		分步二分查找		静态智能体		动态智能体	
		智能体步骤		智能体	Step	智能体步骤		智能体	Step
		Agent	Step	Agent	Step	Agent	Step	Agent	Step
Captain-Agent	带真值	64.7	29.4	25.9	14.1	57.7	22.4	67.1	30.6	**68.2**	**32.9**
	无真值	**63.5**	22.4	24.7	9.4	44.7	17.7	58.8	24.7	61.2	**25.9**
Magentic-One	带真值	58.2	25.2	38.5	13.2	63.7	20.9	67.0	30.8	**68.1**	**33.0**
	无真值	56.0	23.1	37.4	8.8	60.4	15.4	61.5	26.4	**61.5**	**27.5**
SWE-Agent	带真值	**63.6**	29.6	52.3	11.4	61.4	6.8	**63.6**	29.6	**63.6**	**34.1**
	无真值	54.6	22.7	50.0	9.1	56.8	4.6	56.8	27.3	**59.1**	**29.6**
所有平均	带真值	62.2	28.1	38.9	12.9	60.9	16.7	65.9	30.3	**66.7**	**33.3**
	无真值	58.0	22.7	37.4	9.1	54.0	12.5	59.1	26.1	**60.6**	**27.6**

表2：故障归因技术在不同智能体系统下，有真值或无真值场景中的性能（即智能体级准确率和步骤级准确率）比较。我们使用粗体表示最高值，下划线表示次高值。

可观测性配置	智能体	Step
全量输入	**0.62**	**0.28**
无元数据	0.55	0.21
无输入	0.54	0.18
无元数据与输入	0.51	0.16
静态智能体	**0.66**	**0.30**
无元数据	0.57	0.23
无输入	0.56	0.19
无元数据与输入	0.54	0.17

表 3：关于可观测性配置的消融研究。

代理级准确率分别为66.7%和65.9%。动态方法通过受控重新执行和反事实探测主动验证候选故障步骤，从而将步骤级归因提升了10%。这一过程有助于从静态轨迹中过滤出虚假候选项，进而精化归因结果。代理级准确率的提升有限，因为它更依赖于对智能体角色和协调逻辑的理解，而这些信息在完整的静态轨迹中已基本可用。

在所有静态配置中，智能体技术（agentic technique）在几乎所有情况下都取得了最佳性能，这可能是因为该技术能够更好地追踪在多智能体系统失败特征中体现的分析链和工具使用中的责任。此外，在其他静态配置中，All-at-Once 的表现相对优于其他配置。我们将在 4.2.3 节中进行进一步分析。

(2) 消融研究。表3展示了两种代表性技术的消融研究结果，其他技术表现出类似趋势，为简洁起见在此省略。完全可观测性对于准确的故障归因至关重要，且缺少元数据或输入字段都会导致明显的性能下降。此外，步骤级归因比智能体级归因对缺失信息更为敏感，即准确率分别下降76%和22%，这凸显了识别故障步骤所具有的更细粒度、更依赖上下文的特性。

(3) 主干大语言模型的影响。图 3 展示了不同主干大语言模型在性能上的表现。总体而言，Claude-4.5-Sonnet、DeepSeek-R1 和 GPT-4o 在智能体级和步骤级准确性方面均表现出相对较强的性能，这很可能归功于其先进的推理架构和强大的上下文理解能力。相比之下，Qwen3-32B 和 GPT-OSS-20B 的表现较弱，这可能归因于其较小的参数规模以及由此导致的持续多步推理和细粒度因果追踪能力受限。

（4）有无真值。当真值不可用时，所有系统和技术均出现一致的性能下降。这表明，获取参考结果（例如正确答案或测试通过/失败信号）能为归因提供至关重要的指导，尤其是对于更细粒度的步骤归因。智能体方法（尤其是 DynamicAgentic）在缺少真值时表现出相对较小的性能下降。这表明其交互式验证（回放和反事实检查）能够通过测试行为假设来部分弥补缺失的参考信号。

4.2.2 故障与归因模式

(1) 故障责任智能体。图4展示了

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

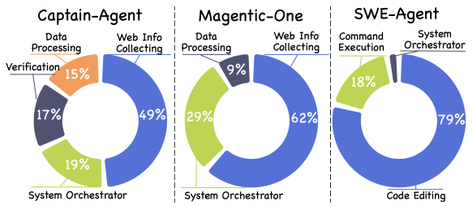


Figure 4: Distribution of failure agent in TraceElephant.

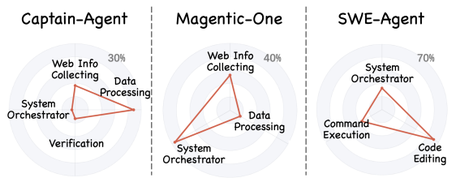


Figure 5: Fine-grained agent-level accuracy.

in our benchmark. Agents responsible for interacting with external environments or performing concrete operations are most prone to errors (almost over 50% of the failures), i.e., agents handling web information collection and browsing in CaptainAgent and Magentic-One, agent directly editing code in SWE-Agent. This might be because these actions depend on dynamic, often noisy external systems (APIs, websites, file systems), where malformed requests, parsing errors, or unexpected outputs can easily occur. The orchestrator/planner agents also represent a non-negligible (18-29%) source of failures. Their mistakes often stem from incorrect task decomposition, sub-optimal agent selection, or flawed coordination logic-errors that may not manifest immediately but can propagate and amplify throughout the execution.

(2) Agent-level Accuracy. Figure 5 further breaks down the predicted agent-level accuracy for these key agent types. For Captain-Agent, data processing agents show high prediction accuracy (53%), likely because their errors (e.g., CSV parsing or script execution failures) leave clear, tool-mediated traces, and web-related agents achieve moderate accuracy (22%), as search API outputs are often ambiguous in natural language. For Magentic-One, the Orchestrator attains higher accuracy (38%) than in Captain-Agent, possibly because its failures frequently arise during visible mid-process interventions, such as re-planning after a browser search stalls, making its responsibility more discernible from output logs.

(3) Decisive Failure Steps. Figure 6 shows the distribution of failure steps over the execution timeline in our benchmark. Failures in the automatically generated system (i.e., Captain-Agent) are dispersed. Since the system constructs and coordinates agent teams on-the-fly for each task, errors can be introduced at multiple points: during agent selection, iterative planning, inter-agent coordination, or tool-calling. Failures in manually crafted systems (i.e., Magentic-One and SWE-Agent) are heavily concentrated in the early steps, tightly linked to the initial task planning and routing decisions made by their central orchestrators.

(4) Step-level Accuracy. Figure 7 demonstrates the step-level accuracy for the ground-truth failure steps divided equally into three segments based on their chronological order. In Captain-Agent and SWE-Agent, step-level accuracy remains relatively stable across all phases, while for Magentic-One, the accuracy is remarkably low in the early phase (8%), moderate in the middle (19%), and high in the late phase (52%). This suggests that failures occurring early in the execution are especially difficult to attribute, whereas those in later stages are more easily identified. A plausible explanation lies in Magentic-One's system design, which often involves extended exploratory and re-planning cycles. Early-phase errors, such as incorrect task decomposition, improper subtask assignment, or flawed initial assumptions, may not manifest immediately and only become visible after subsequent execution fails. In contrast, mid- to late-phase failures often involve tangible inconsistencies, evidence conflicts, or clear execution failures (e.g., mismatched web information), which provide stronger, more localized signals for attribution.

4.2.3 TraceElephant vs. Who&When

Performance under the Output-only Setting. Table 3 demonstrates the performance under *All-at-Once w/o metadata&input*, i.e., only utilizing the step output fields for failure attribution, which is exactly the case as Who&When benchmark (Zhang et al., 2025c). We can see that, solely relying on the output fields, the performance suffers a notable degradation, from 62% to 51% in agent-level accuracy and from 28% to 16% in step-level accuracy. These results are largely consistent with those reported in Who&When (with the same backbone LLM), i.e., agent-level accuracy being 51.1% to 54.3% and step-level accuracy being 12.5% to 13.5%. The key observability gap is therefore not whether intermediate reasoning text appears in the transcript, but whether the actual decision context

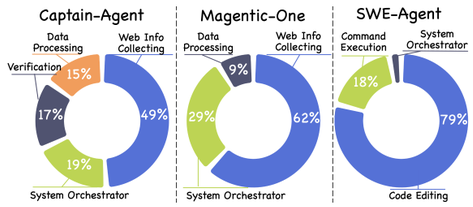


图4：TraceElephant中故障代理的分布。

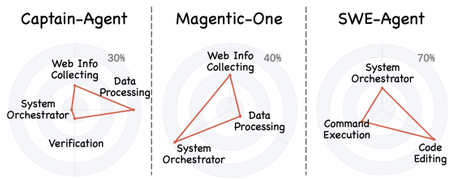


图5：细粒度智能体级准确率。

在我们的基准测试中，易发生故障的智能体类型的分布情况。负责与外部环境交互或执行具体操作的智能体最容易出错（几乎占所有故障的50%以上），例如 CaptainAgent 和 Magentic-One 中处理网络信息收集和浏览的智能体，以及 SWE-Agent 中直接编辑代码的智能体。这可能是因为这些操作依赖于动态且常常存在噪声的外部系统（应用程序接口、网站、文件系统），在这些系统中，畸形请求、解析错误或意外输出很容易发生。编排者/规划者代理也是不可忽视（18-29%）的故障来源。它们的错误往往源于不正确的任务分解、次优的代理选择或存在缺陷的协调逻辑——这些错误可能不会立即显现，但可能会在整个执行过程中传播并被放大。

（2）代理级准确率。图5进一步细分了这些关键智能体类型的预测代理级准确率。对于 Captain-Agent，数据处理代理表现出较高的预测准确率（53%），可能是因为其错误（例如 CSV 解析或脚本执行失败）会留下清晰且由工具中介的执行轨迹；而网络相关代理则达到中等准确率（22%），因为搜索 API 的输出在自然语言中往往具有歧义。对于 Magentic-One，编排器达到了比 Captain-Agent 更高的准确率（38%），这可能是因为其失败往往发生在可见的中途干预阶段，例如在浏览器搜索停滞后的重新规划，从而使得其责任更容易从输出日志中辨识。

(3) 决定性故障步骤。图6展示了基准测试中故障步骤在执行时间线上的分布。自动生成的系统（即 Captain-Agent）中的故障分布较为分散。由于该系统会为每个任务即时构建并协调代理团队，错误可能在多个环节引入：代理选择、迭代规划、代理间协调或工具调用。手动构建的系统（即Magentic-One和SWE-Agent）的故障则高度集中在早期步骤，与其中央编排者所做出的初始任务规划和路由决策紧密相关。

(4) 步骤级准确率。图7展示了将真实故障步骤按照时间顺序均匀划分为三个区段后的步骤级准确率。在Captain-Agent和 SWE-Agent中，步骤级准确率在所有阶段均保持相对稳定；而对于Magentic-One，其在早期阶段的准确率极低（8%），中期适中（19%），后期则很高（52%）。这表明，执行初期发生的故障特别难以进行归因，而后期发生的故障则更容易被识别。一个合理的解释在于Magentic-One的系统设计，该系统通常涉及较长的探索与重新规划周期。早期阶段的错误，如任务分解不正确、子任务分配不当或初始假设存在缺陷，可能不会立即显现，而是在后续执行失败后才变得可见。相比之下，中后期阶段的故障往往涉及实质性不一致、证据冲突或明确的执行失败（例如，网络信息不匹配），这些为归因提供了更强、更局部化的信号。

4.2.3 TraceElephant 对比 Who&When

仅输出设置下的表现。表 3 展示了 All-at-Once w/o metadata&input 设置下的表现，即仅利用步骤输出字段进行故障归因，这正好是 Who&When 基准测试（Zhang et al., 2025c）所面对的情况。我们可以看到，仅依赖输出字段时，性能会出现显著下降，智能体级准确率从 62% 降至 51%，步骤级准确率从 28% 降至 16%。这些结果与 Who&When 中报告的结果（使用相同骨干大型语言模型）大体一致，即智能体级准确率为 51.1% 到 54.3%，步骤级准确率为 12.5% 到 13.5%。因此，关键的可观测性差距并不在于轨迹中是否出现中间推理文本，而在于是否记录了每次大型语言模型调用的实际决策上下文

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

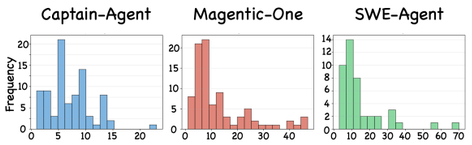


Figure 6: Distribution of failure step in TraceElephant.

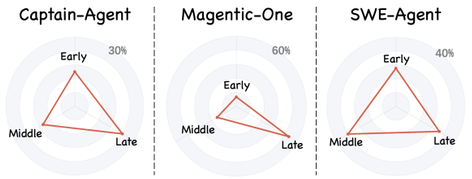


Figure 7: Fine-grained step-level accuracy.

of each LLM call is recorded. Who&When may include intermediate agent outputs, reasoning snippets, or tool-call descriptions, but it does not provide the role-specific prompt, the exact visible history, system-constructed context, agent configuration, or tool/environment information injected into the next prompt. Without these input-side fields, a later output sequence only shows chronological order, not what information each component actually observed when making its decision. TraceElephant records these input contexts explicitly, enabling attribution methods to distinguish failures caused by missing or transformed upstream information from failures caused by the acting component's own reasoning.

Variation in Prompting Strategies. A key difference lies in the comparison across prompting strategies: in our experiments, both agent-level and step-level accuracy achieve the best results under the All-at-Once setting, whereas in Who&When, step-level accuracy peaks under the Step-by-Step setting. This discrepancy may stem from the fact that our execution traces contain more steps (average LLM invocations: 20.5 for Captain-Agent, 29.3 for Magentic-One vs. 9.6 and 28.8 in Who&When), leading to overly long contextual inputs in later interactions of incremental prompts, i.e., exceeding the effective context window of the LLM and thus degrading performance.

We provide more analysis on Appendix A.7.

4.2.4 Implications and Takeaways

Our benchmark and experimental findings offer several actionable insights for the future of failure attribution in MAS.

Architecture-Aware Attribution. As revealed in Section 4.2.2, the types of failure responsible agents and the distribution of decisive failure steps vary significantly across different MAS architectures. The performance of automated attribution itself is highly dependent on agent types and step positions. In practice, this calls for architecture-aware attribution methods that leverage prior knowledge about an MAS's design (e.g., centralized vs. dynamic team formation, tool-heavy vs. planning-heavy workflows) to focus attention on its most vulnerable components and interaction patterns, thereby improving both the efficiency and accuracy of failure attribution.

Enhancing Static Attribution with Advanced Reasoning. Our *Static Agentic* method, while effective, currently employs only basic tool use (e.g., inspecting step I/O, see details in Appendix A.6). Future work can integrate more sophisticated reasoning strategies, such as explicit hypothesis generation and testing cycles, or graph-based reasoning over extracted agent interaction networks. This could help the agent better synthesize long-horizon dependencies and ambiguous failure patterns present in static traces.

Leveraging Dynamic Environment for Deeper Analysis. The current dynamic configuration primarily performs single-step replay and counterfactual checks, which already improve step-level accuracy. The provided executable environment enables more advanced analysis techniques, such as reading code to infer potentially faulty nodes in the system design, reconstructing the actual control-flow structure from execution traces, conducting systematic state-space exploration around failure points, testing system robustness through automated fault injection, or applying causal discovery algorithms that actively intervene on multiple variables to isolate root causes. These measures allow researchers to better understand how error traces arise. As a result, the benchmark is transformed from a passive dataset into an active laboratory for debugging research.

Specializing Models for Attribution via Fine-Tuning. The strong correlation between model reasoning capability (e.g., Claude-4.5, DeepSeek-R1) and attribution accuracy also highlights the need for specialized models. Promisingly, recent work such as GraphTracer (Zhang et al., 2025b) and AgenTracer (Zhang et al., 2025a) shows that task-specific fine-tuning can enable smaller models to surpass larger base models. Future efforts

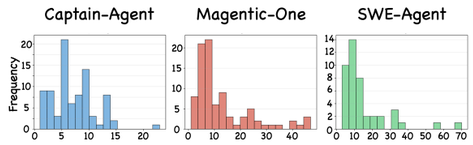


图6：TraceElephant 中故障步骤的分布。

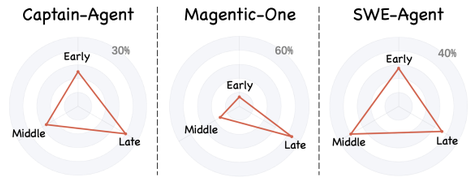


图7：细粒度步骤级准确率。

。Who&When 可能包含中间智能体输出、推理片段或工具调用描述，但它并未提供角色特定提示、确切的可见历史、系统构建的上下文、智能体配置，或注入到下一个提示中的工具/环境信息。缺少这些输入端字段时，后续的输出一段序列只能展示时间顺序，而无法显示每个组件在做出决策时实际观察到的信息。TraceElephant 明确记录这些输入上下文，使归因方法能够区分由缺失或经过转换的上游信息引起的故障，与由行动组件自身推理引起的故障。

提示策略的差异。一个关键区别在于不同提示策略之间的比较：在我们的实验中，无论是智能体级还是步骤级准确率，在全量输入设置下均取得了最佳结果；而在 Who&When 中，步骤级准确率在逐步输入设置下达到峰值。这种差异可能源于我们的执行轨迹包含更多的步骤（大语言模型调用的平均值：Captain-Agent 为 20.5，Magentic-One 为 29.3，而 Who&When 中分别为 9.6 和 28.8），导致增量提示在后续交互中的上下文输入过长，即超出了大语言模型的有效上下文窗口，从而降低了性能。

更多分析请参见附录 A.7。

4.2.4 启示与要点

我们的基准测试和实验结果为多智能体系统中的故障归因提供了若干可操作的见解。

架构感知归因。如第4.2.2节所述，不同多智能体系统架构下，故障责任智能体的类型以及决定性故障步骤的分布存在显著差异。自动化归因自身的性能高度依赖于智能体类型和步骤位置。在实践中，这需要架构感知归因方法，利用多智能体系统设计（例如，集中式与动态团队组建、工具密集型与规划密集型工作流）的先验知识，将注意力集中在其最脆弱的组件和交互模式上，从而提升故障归因的效率和准确性。

增强静态归因中的高级推理。我们的静态智能体方法虽然有效，但目前仅使用基础的工具体系（例如，检查步骤输入/输出，详见附录A.6）。未来的工作可以整合更复杂的推理策略，例如显式的假设生成与验证循环，或基于从智能体交互网络中提取的图结构进行推理。这可以帮助智能体更好地综合静态轨迹中存在的长程依赖关系和模糊的故障模式。

利用动态环境进行深入分析。当前的动态配置主要执行单步重放和反事实检查，这已经提高了步骤级准确性。所提供的可执行环境支持更高级的分析技术，例如阅读代码以推断系统设计中的潜在故障节点、从执行轨迹中重建实际的 control-flow structure、围绕故障点进行系统性的状态空间探索、通过自动化故障注入测试系统鲁棒性，或者应用因果发现算法对多个变量进行主动干预以隔离根本原因。这些措施使研究人员能够更好地理解错误轨迹是如何产生的。因此，该基准测试从一个被动数据集转变为一个用于调试研究的主动实验室。

通过微调实现归因模型的专用化。模型推理能力（例如 Claude-4.5、DeepSeek-R1）与归因准确率之间的强相关性也凸显了对专用模型的需求。令人鼓舞的是，GraphTracer（Zhang et al., 2025b）和 AgenTracer（Zhang et al., 2025a）等近期工作表明，特定任务微调能够使较小模型超越较大基础模型。未来的工作

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

can fine-tune compact models utilizing TraceElephant's traces and running environments, explicitly incorporating structural features (e.g., agent graphs, tool-call sequences) and temporal dependencies as auxiliary training signals to create efficient, attribution-specialized models.

Toward Integrated Debugging Tools. Our findings underscore the practical value of full observability. This motivates the development of integrated debugging tools that automatically capture rich execution traces, visualize agent interaction flows, and suggest potential failure points using attribution models. Embedding such capabilities into MAS development frameworks can significantly reduce debugging overhead.

5 Related Work

LLM-based Multi-Agent Systems. LLM-based Multi-Agent Systems (MASs) have been widely studied as a paradigm for solving complex tasks, where agents interact through natural language, planning modules, and tool usage to perform tasks such as software engineering, question answering, and decision making (Chen et al., 2024; Maldonado et al., 2024; Zhang et al., 2025d). Existing work has explored different system architectures, including centralized planning, decentralized coordination, and hybrid designs, as well as mechanisms for role assignment and communication (Wu et al., 2024; Li et al., 2023; Fourney et al., 2024; Song et al., 2025; Team, 2025; Jin et al., 2025). As MAS are deployed in increasingly complex and long-horizon tasks, failures become more difficult to diagnose and failure attribution is even challenging.

Failure Attribution in MAS. There are several techniques for failure attribution in MASs. For example, FAMAS (Ge et al., 2025) conducted spectrum analysis on multiple execution trajectories collected by repeatedly replaying a failed task. AgentTracer (Zhang et al., 2025a) proposed a lightweight model trained via multi-granular reinforcement learning to jointly optimize step-level and agent-level attribution accuracy. GraphTracer (Zhang et al., 2025b) modeled agent interactions as information dependency graphs, and traced causal information flows. ECHO (Banerjee et al., 2025) combined hierarchical context representation, objective analysis-based evaluation, and consensus voting to improve failure attribution accuracy. To reliably evaluate these existing techniques and support the development of new ones, high-quality benchmarks like ours are essential.

6 Conclusion

The inherent complexity and non-deterministic interactions in LLM-based MASs make failure attribution a core challenge for ensuring operational reliability and facilitating targeted debugging. To support this, we introduce TraceElephant, a benchmark for failure attribution of LLM-based MASs under full execution observability. Experiments show that full observability significantly improves attribution performance, and dynamic replay further enhances attribution capability. This work lays a practical foundation for future failure attribution research and promotes evaluation practices that mirror real-world debugging.

7 Limitations

This work focuses on failure attribution under developer-facing settings using execution traces collected from a limited set of representative multi-agent systems. While this scope does not cover all possible system architectures or black-box usage scenarios, it reflects realistic debugging conditions in which full execution traces are available. As this study is conducted on only three MASs, some of our findings may not generalize to all existing or future systems. However, the selected systems, i.e., Captain-Agent, Magentic-One, and SWE-Agent, are intentionally diverse in their design paradigms, covering dynamic team assembly, centralized orchestration, and specialized software engineering workflows. This deliberate diversity enhances the representativeness of our benchmark and mitigates the risk of architecture-specific bias.

References

Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, and 1 others. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.

Anthropic. 2025. Introducing Claude Sonnet 4.5. 2025-01-05.

Adi Banerjee, Anirudh Nair, and Tarik Borogovac. 2025. Where did it all go wrong? a hierarchical look into multi-agent error attribution. *arXiv preprint arXiv:2510.04886*.

可以利用 TraceElephant 的执行轨迹和运行环境对紧凑模型进行微调，显式地将结构特征（例如智能体图、工具调用序列）和时序依赖作为辅助训练信号，以创建高效、归因专用模型。

迈向集成调试工具。我们的研究结果凸显了全面可观测性的实际应用价值。这激发了开发集成调试工具的动机，这些工具能够自动捕获丰富的执行轨迹、可视化智能体交互流，并使用归因模型建议潜在的故障点。将这些功能嵌入到多智能体系统开发框架中，可以显著降低调试开销。

5 相关工作

基于大语言模型的多智能体系统。基于大语言模型的多智能体系统（MASs）已被广泛研究，作为一种解决复杂任务的范式，其中智能体通过自然语言、规划模块和工具使用来执行诸如软件工程、问答和决策等任务（Chen et al., 2024; Maldonado et al., 2024; Zhang et al., 2025d）。现有工作已探索了不同的系统架构，包括集中式规划、去中心化协调和混合设计，以及角色分配和通信机制（Wu et al., 2024; Li et al., 2023; Fourney et al., 2024; Song et al., 2025; Team, 2025; Jin et al., 2025）。随着多智能体系统被部署在日益复杂和长视野的任务中，故障变得更难诊断，故障归因也极具挑战性。

多智能体系统中的故障归因。在多智能体系统中有多种故障归因技术。例如，FAMAS (Ge et al., 2025) 通过对多次重放失败任务所收集的多条执行轨迹进行频谱分析来识别故障源。AgentTracer (Zhang et al., 2025a) 提出了一种轻量级模型，通过多粒度强化学习进行训练，以联合优化步骤级和智能体级的归因准确率。GraphTracer (Zhang et al., 2025b) 将智能体间的交互建模为信息依赖图，并追踪因果信息流。ECHO (Banerjee et al., 2025) 结合了层次化上下文表示、基于目标分析的评估方法以及共识投票机制，以提升故障归因准确率。至要可靠地评估这些现有技术并支持新技术的开发，高质量的基准测试（如我们的）至关重要。

6 结论

基于大语言模型的多智能体系统的固有复杂性和非确定性交互，使得故障归因成为确保运行可靠性并促进针对性调试的核心挑战。为此，我们提出了TraceElephant，这是一个在完整执行可观测性下对基于大语言模型的多智能体系统进行故障归因的基准。实验表明，完全可观测性显著提升了归因性能，而动态重放则进一步增强了归因能力。本工作为未来的故障归因研究奠定了实践基础，并推动了反映真实世界调试过程的评估实践。

7 局限性

本工作聚焦于在面向开发者的设置下，利用从有限的一组具有代表性的多智能体系统中收集的执行轨迹进行故障归因。尽管这一范围并未涵盖所有可能的系统架构或黑盒使用场景，但它反映了在可获得完整执行轨迹的实际调试条件下的情况。由于本研究仅在三个多智能体系统上进行，我们的部分发现可能无法推广到所有现有或未来的系统。然而，所选取的系统，即 Captain-Agent、Magentic-One 和 SWE-Agent，在设计范式上刻意保持了多样性，涵盖了动态团队组装、集中式编排以及专业软件工程工作流。这种刻意的多样性增强了我们基准测试的代表性，并降低了架构特定偏差的风险。

参考文献

桑德尼·阿加瓦尔、拉玛·艾哈迈德、杰森·艾、山姆·奥特曼、安迪·阿普尔鲍姆、埃德温·阿布斯、拉胡尔·K·阿罗拉、俞白、鲍文·贝克、鲍海明及其他1位作者。2025年。gpt-oss-120b和gpt-oss-20b模型卡片。arXiv预印本 arXiv:2508.10925。

Anthropic. 2025年. 推出Claude Sonnet 4.5. 2025-01-05.

AdiBanerjee、阿尼尔鲁德·奈尔和塔里克·博罗戈瓦茨。2025年。《哪里出了差错？对多智能体错误归因的层级化审视》。arXiv 预印本 arXiv:2510.04886。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Robert N Charette. 2005. Why software fails [software failure]. *IEEE spectrum*, 42(9):42–49.

Shuaihang Chen, Yuanxing Liu, Wei Han, Weinan Zhang, and Ting Liu. 2024. A survey on LLM-based multi-agent system: Recent advances and new frontiers in application. *arXiv preprint arXiv:2412.17481*.

Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang, Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. Magentic-One: A generalist multi-agent system for solving complex tasks. *Preprint*, arXiv:2411.04468.

Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. 2025. Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis. *Preprint*, arXiv:2509.13782.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. DeepSeek-R1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Sheng Jin, Haoming Wang, Zhiqi Gao, Yongbo Yang, Bao Chunjia, and Chengliang Wang. 2025. Evolution in simulation: Ai-agent school with dual memory for high-fidelity educational dynamics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5843–5857.

Klaus Krippendorff. 2018. *Content analysis: An introduction to its methodology*. Sage publications.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008.

Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9.

Diego Maldonado, Edison Cruz, Jackeline Abad Torres, Patricio J Cruz, and Silvana del Pilar Gamboa Benitez. 2024. Multi-agent systems: A survey about its components, framework and workflow. *IEEE Access*, 12:80950–80975.

Ghassan Misherghi and Zhendong Su. 2006. HDD: hierarchical delta debugging. In *Proceedings of the 28th international conference on Software engineering*, pages 142–151.

OpenAI. 2024. GPT-4o mini: advancing cost-efficient intelligence. 2025-01-05.

Linxin Song, Jiale Liu, Jieyu Zhang, Shaokun Zhang, Ao Luo, Shijian Wang, Qingyun Wu, and Chi Wang. 2025. Adaptive in-conversation team building for language model agents. *Preprint*, arXiv:2405.19425.

MiroMind AI Team. 2025. MiroFlow: A high-performance open-source research agent framework. https://github.com/MiroMindAI/MiroFlow.

W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2024. AutoGen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-Agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA. Curran Associates Inc.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2025a. AgenTracer: Who is inducing failure in the llm agentic systems? *arXiv preprint arXiv:2509.03312*.

Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. 2025b. GraphTracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search. *arXiv preprint arXiv:2510.10581*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and 1 others. 2025c. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. *ICML 2025 Spotlight*.

Xueqiang Zhang, Xiaofei Dong, Yiru Wang, Dan Zhang, and Feng Cao. 2025d. A survey of multi-ai agent collaboration: Theories, technologies and applications. In *Proceedings of the 2nd Guangdong-Hong Kong-Macao Greater Bay Area International Conference*

罗伯特·N·沙雷特。2005。《为何软件会失效》 [软件故障]。IEEE 电路与系统, 42(9):42–49。

陈帅航, 刘元星, 韩伟, 张伟南, 和刘婷. 2024. A survey on 大型语言模型- based 多智能体系统: Recent advances and new frontiers in application. arXiv 预印本 arXiv:2412.17481.

亚当·福尼, 加根·班萨尔, 侯赛因·莫扎纳尔, 谭成, 爱德华多·萨利纳斯, 朱尔康, 弗里德里克·尼特纳, 格蕾丝·普罗宾斯廷, 格里芬·巴斯曼, 杰克·格利茨, 雅各布·阿尔贝尔, 彼得·张, 里基·洛伊德, 罗伯特·韦斯特, 维克多·迪比亚, 艾哈迈德·阿瓦达拉, 埃切·卡马尔, 拉法·霍斯恩, 和萨拉玛·阿梅尔希. 2024. Magentic-One: A 通用多智能体系统 for solving 复杂任务. 预印本 , arXiv:2411.04468.

葛宇, 谢丽娜, 李仲, 裴宇, 和张天. 2025年. Who is introducing the failure? automatically attributing failures of 多智能体系统 via spec- trum analysis. 预印本, arXiv:2509.13782.

郭达雅, 杨德建, 张浩威, 宋俊晓, 王沛怡, 朱启豪, 徐润心, 张若宇, 马世荣, 毕晓， 以及其他1人。2025年。DeepSeek-R1通过强化学习激励大语言模型的推理。自然, 645(8081):633–638。

郭泰成, 陈秀英, 王亚琪, 常瑞迪, 裴世超, 尼特什·V·乔拉, 奥拉夫·维斯特， 以及 Xi-angliang Zhang。2024年。基于大语言模型的多智能体：进展与挑战综述。arXiv 预印本 arXiv:2402.01680。

亚伦·赫斯特, 亚当·莱勒, 亚当·P·古彻, 亚当·佩尔曼, 阿迪蒂亚·拉梅什, 艾登·克拉克, AJ·奥斯特罗, 阿基拉·韦利欣达, 艾伦·海耶斯, 亚历克·拉德福德， 以及其他1人。 2024年。GPT-4o 系统卡。arXiv 预印本 arXiv:2410.21276。

金胜， 王浩铭， 高志奇， 杨永波， 包春佳， 和王成梁。2025年。模拟中的演化：具有双重记忆的AI智能体学校用于高保真教育动态。收录于《2025年计算语言学协会发现：EMNLP 2025》， 第5843–5857页。

克劳斯·克里彭多夫。*2018*。内容分析：方法论导论。*Sage* 出版社。

Guohao Li， Hasan Hammoud， Hani Itani, Dmitrii Khizbullin， 和Bernard Ghanem。2023。Camel：用于探索大型语言模型社会“心智”的沟通型智能体。《神经信息处理系统进展》， 36:51991–52008。

Xinyi Li， Sai Wang， Siqi Zeng， Yu Wu， 和Yi Yang。2024。基于大型语言模型的多智能体系统综述：工作流、基础设施与挑战。Vicinagearth， 1(1):9。

Diego Maldonado、Edison Cruz、Jackeline Abad Torres、Patricio J Cruz 和 Silvana del Pilar Gamboa Benítez. 2024. 多智能体系统：关于其组件、框架和工作流的综述. IEEE Access, 12:80950–80975.

Ghassan Misherghi and Zhendong Su. 2006. HDD：层级差异调试。载于《第28届国际软件工程大会论文集》， 第142–151页。OpenAI. 2024. GPT-4o mini：推进高性价比智能。2025年-01-05。Linxin Song, Jiale Liu, Jieyu Zhang, Shaokun Zhang, Ao Luo, Shijian Wang, Qingyun Wu, and Chi Wang. 2025. 语言模型代理的自适应对话中团队构建。预印本, arXiv:2405.19425。MiroMind AI 团队. 2025年。MiroFlow：一个高性能开源研究智能体框架。https://github.com/MiroMindAI/MiroFlow。

W Eric Wong、Ruizhi Gao、Yihao Li、Rui Abreu 和 Franz Wotawa. 2016. 软件故障定位综述. IEEE Transactions on Software Engineering, 42(8):707–740.

Qingyun Wu, 加根·班萨尔, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2024. Auto- Gen: Enabling next-gen 大型语言模型 applications via multi- 智能体 conversations. In First Conference on Lan- guage Modeling.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025年. Qwen3 technical report. arXiv 预印本 arXiv:2505.09388.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kil- ian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024年. SWE-Agent: agent-computer in- terfaces enable automated software engineering. In Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24, Red Hook, 纽约, 美国. Curran Associates Inc.

Guibin Zhang,Junhao Wang, Junjie Chen,Wangchun- s hu Zhou, Kun Wang, and Shuicheng Yan. 2025a. AgenTracer: Who is inducing failure in the 大语言模型智能体 系统? arXiv 预印本 arXiv:2509.03312.

Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. 2025b. GraphTracer: Graph-guided failure 轨迹 in 大语言模型智能体 for robust multi-turn deep search. arXiv 预印本 arXiv:2510.10581.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen， 以及其他 1 人。2025c. Which agent causes task failures and when? on 大型语言模型多智能体系统的自动故障归因. ICML 2025 Spotlight.

Xueqiang Zhang, Xiaofei Dong, Yiru Wang, Dan Zhang, and Feng Cao. 2025d. A survey of multi-ai agent col- laboration: Theories, technologies and applications. 载于第二届中国香港-澳门大湾区国际会议

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

on Digital Economy and Artificial Intelligence, pages 1875–1881.

Alice X Zheng, Michael I Jordan, Ben Liblit, Mayur Naik, and Alex Aiken. 2006. Statistical debugging: simultaneous identification of multiple bugs. In *Proceedings of the 23rd international conference on Machine learning*, pages 1105–1112.

Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D Ernst, and Lu Zhang. 2019. An empirical study of fault localization families and their combinations. *IEEE Transactions on Software Engineering*, 47(2):332–347.

A Appendix

A.1 Detailed Problem Definition

Multi-Agent Execution and Observability. We consider an LLM-based multi-agent system (MAS) $\mathcal{M}$ composed of a finite set of agents $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ that collaboratively perform a task τ. The system executes in discrete steps. At each step, a single agent is selected to act, possibly through a dispatcher, router, or planning component that is part of the system. At step t, the acting agent is selected as $a(t) = g(h_{t-1})$, where $g(\cdot)$ represents the system-level agent selection mechanism and h_{t-1} denotes the system context at step $t-1$, summarizing the execution history. The selected agent receives an input $x_t = \phi(h_{t-1})$, and produces an output $y_t = \pi_{a(t)}(x_t)$, where $\pi_{a(t)}$ denotes the policy of agent $a(t)$. The system context is then updated as $h_t = u(h_{t-1}, a(t), x_t, y_t)$. The execution terminates after T steps, producing an execution trace $\mathcal{T} = \langle s_1, s_2, \ldots, s_T \rangle$. Each execution step s_t is associated with a structured observable record $o_t = \big(x_t,\ y_t,\ a(t),\ \textit{step_id}_t,\ \textit{agent_id}_t\ \big)$.

Task Outcome and Failure Attribution. Given a completed execution trace $\mathcal{T}$, the system produces a task outcome. We define a task outcome function $\Omega(\mathcal{T}) \in \{0, 1\}$, where $\Omega(\mathcal{T}) = 1$ indicates successful task completion and $\Omega(\mathcal{T}) = 0$ indicates failure. Failure attribution aims to identify the agent and the execution step responsible for a task failure based on observable execution traces. We define failure attribution at two levels. **Step-level attribution.** Let $\mathcal{T}_{\leq t}$ denote the prefix of the execution trace up to step t. We define the set of feasible continuations from $\mathcal{T}_{\leq t}$ under system $\mathcal{M}$ as $\mathcal{C}(\mathcal{T}_{\leq t}) = \{\mathcal{T}' \mid \mathcal{T}' \text{ is a valid continuation from } \mathcal{T}_{\leq t}\}$. Failure is said to become inevitable at step t if all feasible continuations result in failure: $\forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}),\ \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0$. The failure step is defined as the earliest inevitable step $t^* = \min \Big\{ t \in \{1, \ldots, T\} \mid \forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}),\ \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0 \Big\}$. **Agent-level attribution.** Given an execution trace $\mathcal{T}$ and its failure step t^*, agent-level attribution identifies the agent whose action at the failure step gives rise to the task failure. Formally, the failure-responsible agent is defined as $a^* = \arg\max_{a \in \mathcal{A}} \mathbb{I}[\exists\, s_{t^*} \in \mathcal{T} \text{ s.t. } a(t^*) = a]$, where $a(t^*)$ denotes the agent selected by the system to act at step t^* in the execution trace $\mathcal{T}$.

A.2 Details about Data Sources

LLM-based Multi-Agent Systems. We collect execution traces from three representative agentic systems: *Captain-Agent*, *Magentic-One*, and *SWE-Agent*. Captain-Agent (Song et al., 2025) follows an automated system construction paradigm, where a meta-controller dynamically assembles and coordinates a team of agents for each task through iterative planning and execution. In contrast, Magentic-One (Fourney et al., 2024) adopts a manually specified architecture, in which an orchestrator governs a fixed set of specialized agents. SWE-Agent (Yang et al., 2024) represents a single-agent tool-centric scaffold for software engineering tasks, where planning, orchestration, and tool invocation act as functional components for attribution. For each system, we use the official implementation and retain the original agent definitions, prompts, and tool interfaces.

Task Sources. For both *Captain-Agent* and *Magentic-One*, execution traces are collected on tasks from *GAIA* and *AssistantBench*, reflecting general-purpose agentic problem solving involving multi-step reasoning, information gathering, and tool use. *GAIA* tasks typically require agents to decompose a question, retrieve information from multiple sources, and integrate intermediate results through a sequence of reasoning steps. *AssistantBench* tasks further emphasize interactive tool use and sequential decision making, where errors may arise from incorrect tool selection, incomplete information propagation, or early planning mistakes. For *SWE-Agent*, traces are collected on *SWE-Bench Verified* tasks, which focus on software engineering scenarios such as code navigation, modification, and validation. These tasks often involve long execution traces with repeated interactions between language model agents and external development environments. Failures in this setting commonly

《数字经济与人工智能》论文集，第*1875–1881*页。

Alice X Zheng, Michael I Jordan, Ben Liblit, Mayur Naik, and Alex Aiken. 2006. Statistical debugging: simultaneous identification of multiple bugs. In Proceedings of the 23rd international conference on Machine learning, pages 1105–1112.

邹大明，梁晶晶，熊英飞，Michael D Ernst，张禄。2019年。故障定位家族及其组合的实证研究。IEEE软件工程学报，47(2):332–347。

附录 A

A.1 详细的问题定义

多智能体执行与可观测性。我们考虑一个由有限智能体集合 $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ 组成的大型语言模型多智能体系统（MAS）$\mathcal{M}$，这些智能体协作执行任务 τ。系统以离散步骤执行。在每个步骤中，选择一个智能体执行操作，可能通过系统的一部分，即调度器、路由器或规划组件。在步骤 t，被选中的行动智能体为 $a(t) = g(h_{t-1})$，其中 $g(\cdot)$ 表示系统级智能体选择机制，h_{t-1} 表示步骤 $t-1$ 处的系统上下文，概括执行历史。被选中的智能体接收输入 $x_t = \phi(h_{t-1})$，并产生输出 $y_t = \pi_{a(t)}(x_t)$，其中 $\pi_{a(t)}$ 表示智能体 $a(t)$ 的策略。随后系统上下文更新为$h_t = u(h_{t-1}, a(t)\ x_t, y_t)$。系统在 T 个步骤后终止执行，产生执行轨迹 $\mathcal{T} = \langle s_1, s_2, \ldots, s_T \rangle$。每个执行步骤 s_t 都与一条结构化可观测记录$o_t = \big(x_t,\ y_t,\ a(t)$、步骤 _ 标识符 $_t$、智能体 _ 标识符 $_t\ \big)$ 相关联。

任务结果与故障归因。给定一条已完成的执行轨迹 $\mathcal{T}$，系统产生一个任务结果。我们定义任务结果函数 $\Omega(\mathcal{T}) \in \{0, 1\}$，其中 $\Omega(\mathcal{T}) = 1$ 表示任务成功完成，而 $\Omega(\mathcal{T}) = 0$表示失败。故障归因旨在根据可观察的执行轨迹识别导致任务失败的智能体和执行步骤。我们在两个层级上定义故障归因。步骤级归因。设 $\mathcal{T}_{\leq t}$表示执行轨迹直到步骤 t的前缀。在系统 $\mathcal{M}$ 下，我们将从 $\mathcal{T}_{\leq t}$ 出发的可行延续集合定义为 $\mathcal{C}(\mathcal{T}_{\leq t})$ =，其中 $\{\mathcal{T}' \mid \mathcal{T}'$ 是从 $\mathcal{T}_{\leq t}\}$. 出发的有效延续。当所有可行延续都导致失败时，我们称失败在步骤 t 变得不可避免： $\forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}), \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0$. 故障步骤被定义为最早的不可避免步骤 $t^* = \min \Big\{ t \in \{1, \ldots, T\} \mid \forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}), \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0 \Big\}$. 智能体级归因。给定执行轨迹 $\mathcal{T}$及其故障步骤 t^*，智能体级归因识别在故障步骤上其操作导致任务失败的那个智能体。形式上，故障责任代理被定义为 $a^* = \arg\max_{a \in \mathcal{A}} \mathbb{I}[\exists\, s_{t^*} \in \mathcal{T}$，约束条件为 $a(t^*) = a]$，其中 $a(t^*)$ 表示在执行轨迹 $\mathcal{T}$中步骤 t^* 由系统选定执行操作的智能体。

A.2 关于数据来源的详细说明

基于大语言模型的多智能体系统。我们从三个代表性的智能体系统收集执行轨迹：Captain-Agent、Magentic-One 和 SWE-Agent。Captain-Agent（Song et al., 2025年）遵循自动化系统构建范式，其中元控制器通过迭代规划与执行，动态地组装并协调由智能体组成的团队来处理每个任务。相比之下，Magentic-One（Fourney et al., 2024）采用手动指定的架构，其中编排器管理一组固定的专业代理。SWE-Agent（Yang et al., 2024）代表了面向软件工程任务的单智能体工具中心脚手架，其中规划、编排和工具调用作为用于归因的功能组件。对于每个系统，我们使用官方实现，并保留原始的代理定义、提示词和工具接口。

任务来源。对于 Captain-Agent 和 Magentic-One，执行轨迹是在 GAIA 和 AssistantBench 的任务上收集的，这反映了涉及多步推理、信息收集和工具使用的通用代理问题解决。GAIA 任务通常要求智能体分解问题、从多个来源检索信息，并通过一系列推理步骤整合中间结果。Assistant- Bench 任务进一步强调交互式工具使用和顺序决策，其中错误可能源于错误的工具选择、不完整的信息传播或早期规划错误。对于 SWE-Agent，执行轨迹是在 SWE-Bench Verified 任务上收集的，这些任务侧重于软件工程场景，例如代码导航、修改和验证。这些任务通常涉及长执行轨迹，其中语言模型代理与外部开发环境之间存在反复交互。在此场景中，失败通常

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

manifest as incorrect localization of code changes, incomplete fixes, or mismatches between generated patches and test requirements, resulting in qualitatively different failure patterns compared to general-purpose reasoning tasks.

Run Configuration. For each system-task pair, we execute the system under a fixed configuration θ and assign a unique run identifier ρ. To ensure stable tool invocation behavior while retaining a controlled degree of stochastic diversity, we set the decoding temperature as 0.3 across all runs and repeat each system-task pair for multiple trials.

Trace Statistics. Across all systems and task sources, a total of 380 execution traces are collected after data cleaning, among which 220 traces result in task failures and are included as benchmark instances. The number of failed traces varies across systems and task sources, reflecting differences in task difficulty, execution length, domain constraints, and system specialization. Rather than enforcing a uniform failure rate, TraceElephant preserves the naturally occurring failure distributions observed during execution. This design choice allows failure attribution methods to be evaluated under realistic conditions, where failures may stem from diverse causes and arise at different stages of execution. Each failed run is treated as an independent attribution instance, capturing variability introduced by language model decoding and tool interactions.

A.3 Details about Trace Collection

To enable systematic failure attribution across heterogeneous MAS, TraceElephant adopts an automated trace collection pipeline that adapts different systems into a unified execution logging interface without modifying their original implementations.

LLM API Middleware. At the core of the trace collection pipeline is a lightweight LLM API middleware that mediates all interactions between a running MAS and its underlying LLMs. Instead of directly communicating with an external LLM service, the MAS is configured to send LLM requests through a middleware. It serves as a transparent proxy between a running multi-agent system and its underlying large language model. All LLM requests issued by agents are intercepted by the middleware, forwarded to a user-specified backend model, and logged together with the corresponding responses. This design enables complete observability of LLM-mediated decision making without modifying agent logic, prompts, or system control flow. Instead of directly communicating with an external LLM service, the multi-agent system is configured to route all LLM requests through the middleware. For each request, the middleware records the request payload (including messages and decoding parameters), forwards it to the backend model, receives the response, and returns the response to the calling agent. Since tool invocations in modern MASs are typically triggered by LLM-generated outputs, the middleware additionally captures LLM-mediated tool interactions when such information is observable at runtime. This includes the selected tool name, tool input arguments, and execution results. As a result, both language-model reasoning and subsequent tool-mediated actions are recorded in a unified, system-agnostic manner.

Trace Pre-processing. We apply lightweight pre-processing to raw traces to expose necessary information in a structured manner while preserving their original form as much as possible. Specifically, we use regular-expression matching to perform two annotations: (1) extracting and tagging agent names in the traces, and (2) categorizing step outputs as either plain LLM responses or tool-mediated interactions.

Agent Name Extraction. Agent names are extracted directly from execution-time observations using pattern matching, without introducing additional semantic inference. In many MAS implementations, agent identities are explicitly referenced in message headers or system-generated markers. For example, given the message:

```
Planner: We need to search for
relevant documentation.
```

The agent name `Planner` is extracted and associated with the current execution step.

Output type categorization. Each step output is categorized as either a plain LLM response or a tool-mediated interaction. This distinction is determined by matching tool invocation patterns in the output text. For instance, the output

```
Action: search_web
Input: “Python regex example”
```

is classified as a tool-mediated step.

Step and Agent Indexing. Each recorded trace is assigned a monotonically increasing *step_id* according to its chronological order in the execution

表现为代码变更的错误定位、不完整的修复，或生成的补丁与测试要求之间的不匹配，与通用推理任务相比，导致了性质不同的失败模式。

运行配置。对于每个系统-任务对，我们在固定配置θ 下执行系统，并分配一个唯一的运行标识符 ρ。为了在保留受控程度的随机多样性的同时确保稳定的工具调用行为，我们在所有运行中将解码温度设置为0.3，并对每个系统-任务对重复多次试验。

轨迹统计。在所有系统和任务来源中，数据清洗后共收集了380条执行轨迹，其中220条执行轨迹导致任务失败，并被纳入基准实例。失败执行轨迹的数量在不同系统和任务来源之间存在差异，反映了任务难度、执行长度、领域约束和系统专门化方面的差异。TraceElephant并非强制采用均匀失败率，而是保留了执行过程中观察到的自然发生的失败分布。这一设计选择使得故障归因方法能够在现实条件下进行评估，在这些条件下，故障可能源于多种多样的原因，并发生在执行的不同阶段。每次失败的运行被视为一个独立的归因实例，捕捉由语言模型解码和工具交互引入的变异性。

A.3 轨迹收集详情

为了实现跨异构多智能体系统的系统性故障归因，TraceElephant 采用了一种自动化轨迹收集流水线，该流水线能够将不同系统适配到统一执行日志接口，同时无需修改其原始实现。

LLM API 中间件。轨迹收集流水线的核心是一个轻量级的 LLM API 中间件，它中介运行中的多智能体系统及其底层大语言模型之间的所有交互。多智能体系统不直接与外部 LLM 服务通信，而是被配置为通过中间件发送 LLM 请求。它在运行中的多智能体系统及其底层大语言模型之间充当透明代理。智能体发出的所有 LLM 请求都会被中间件拦截，并转发到用户指定的后端模型，连同相应的响应一起被记录。此设计能够实现对 LLM 介导的决策制定的完全可观察性，而不会修改智能体逻辑、提示词或系统控制流。多智能体系统不直接与外部 LLM 服务通信，而是被配置为将所有 LLM 请求通过中间件路由。对于每个请求，中间件记录请求负载（包括消息和解码参数），将其转发到后端模型，接收响应，并将响应返回给调用方智能体。由于现代多智能体系统中的工具调用通常由大语言模型生成的输出触发，当运行时可以观察到此类信息时，中间件还会捕获 LLM 介导的工具交互。这包括所选的工具名称、工具输入参数以及执行结果。因此，语言模型推理和后续的工具中介操作都以统一且与系统无关的方式被记录。

轨迹预处理。我们对原始轨迹应用轻量级预处理，以结构化方式暴露必要信息，同时尽可能保留其原始形式。具体而言，我们使用正则表达式匹配执行两项标注：(1) 提取并标记执行轨迹中的智能体名称，以及 (2) 将步骤输出分类为普通LLM响应或工具中介交互。

智能体名称提取。智能体名称通过模式匹配直接从执行期间的观测中获取，不引入额外的语义推断。在许多多智能体系统（MAS）的实现中，智能体身份在消息头或系统生成的标记中被显式引用。例如，给定如下消息：

规划器：我们需要搜索相关文档。

提取智能体名称“规划器”，并将其与当前执行步骤关联。

输出类型分类。每个步骤的输出被归类为纯大型语言模型响应或工具介导交互。这种区分是通过匹配输出文本中的工具调用模式来确定的。例如，输出

操作：搜索_Web 输入：“Python 正则表达式示例”

被归类为工具介导步骤。

步骤与智能体索引。每条记录的轨迹都会根据其执行中的时间顺序，被分配一个单调递增的步骤_id。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

trace. Agent identifiers (*agent_id*) are assigned based on the first occurrence of each agent name and remain consistent within a trace. Each step is linked to both the corresponding *agent_id* and the *agent name*.

A.4 Example Execution Trace

We provide a concrete example to illustrate how execution traces are recorded in TraceElephant. The example corresponds to a general travel-planning task executed by a multi-agent system.

Task. *"How should a first-time visitor plan a 3-day trip to Walt Disney World in Florida?"* This task requires multi-step reasoning, external information retrieval, and verification across multiple constraints, making it representative of realistic agentic workloads.

Trace Metadata. Each execution trace begins with trace-level metadata that provides global context for the execution.


```
{
  "trace_metadata": {
    "task_id": "WDW-TRAVEL-001",
    "task_instruction": "How should a
        first-time visitor plan a 3-
       day trip to Walt Disney World
        in Florida?",
    "system_name": "Magentic-One",
    "agent_configuration": {
      "agents": [
        "PlannerAgent",
        "SearchAgent",
        "ItineraryAgent",
        "VerifierAgent"
      ],
      "prompts": ["..."],
      "tools": ["web_search"]
    },
    "system_architecture": {
      "description": "A centrally
          orchestrated multi-agent
          system with role-
          specialized agents.",
      "code": ["orchestrator.py", "
          agents/*.py"]
    }
  }
}
```


The metadata records the task, system identity, agent composition, and architectural information. Besides for failure attribution, these fields can also provide essential context for interpreting execution behavior and comparing traces across systems.

Step-level Records. The execution trace consists of an ordered sequence of step-level records, each corresponding to a single agent action. *Step 1: Task Decomposition*


```
{
  "step_id": 1,
  "agent_id": 0,
  "agent_name": "PlannerAgent",
  "input_context": "Task: Plan a
      three-day Walt Disney World
      visit for a family.",
  "output_content": "I will break the
       task into park selection,
      ticket options, and visit order
      ."
}
```


The planner interprets the user request and decomposes it into actionable sub-tasks.

Step 2: Identifying Disney Theme Parks


```
{
  "step_id": 2,
  "agent_id": 1,
  "agent_name": "SearchAgent",
  "input_context": "Find the main
      theme parks at Walt Disney
      World.",
  "output_content": "Searching
      official Disney sources.",
  "tool_logs": {
    "tool_name": "web_search",
    "input_arguments": "Walt Disney
        World theme parks",
    "outputs": "Magic Kingdom, EPCOT,
        Hollywood Studios, and
        Animal Kingdom.",
    "execution_status": "success"
  }
}
```


This step involves an external tool invocation. The raw tool interaction is preserved verbatim in *tool_logs*. Please note that for each step of the output, there must be an *output_content*, and only when there is a tool call will there be a *tool_logs*.

Step 3: Retrieving Ticket Options

轨迹。智能体标识符（agent_id）根据每个智能体名称的首次出现进行分配，并在同一轨迹内保持一致。每个步骤都同时关联对应的agent_id和智能体名称。

A.4 示例执行轨迹

我们提供了一个具体示例，以说明执行轨迹如何在 TraceElephant 中被记录。该示例对应于由多智能体系统执行的一般性旅行规划任务。

任务。“首次前往佛罗里达州华特迪士尼世界的游客应如何规划为期三天的行程？”该任务需要多步推理、外部信息检索以及对多个约束条件的验证，使其成为典型智能体工作负载的代表。

轨迹元数据。每条执行轨迹都以轨迹级元数据开始，为执行提供全局上下文。

{ "trace_metadata": { "task_id": "WDW - TRAVEL -001", "task_instruction": "第一次到访佛罗里达州华特迪士尼世界的游客应如何规划为期3天的行程？ ", "system_name": "Magentic - One", "agent_configuration": { "agents": [" PlannerAgent", "SearchAgent", " ItineraryAgent", "VerifierAgent",],"prompts": ["..."],", "tools": ["web_search"]}, "system_ architecture": { "description": "由中央编排器协调的多智能体系统，配备角色专用的专业代理。", "code": ["orchestrator . py", "agents /*. py"]} } }

元数据记录了任务、系统标识、智能体组成以及架构信息。除了用于故障归因外，这些字段还可以为解释执行行为和跨系统比较执行轨迹提供必要的上下文。

步骤级记录。执行轨迹由一系列有序的步骤级记录组成，每条记录对应单个智能体动作。步骤 1：任务分解

{ " step_id ": 1, " agent_id ": 0, " agent_name ": " PlannerAgent " , " input_context ": " 任务：为一家人规划为期三天的华特迪士尼世界之旅。", " output_content ": "我将把任务分解为公园选择、门票选项和游览顺序" }

规划器解析用户请求，并将其分解为可执行的子任务。

步骤 *2*：识别迪士尼主题公园

{ " step_id ": 2, " agent_id ": 1, " agent_name ": " SearchAgent " , " input_context ": " 查找华特迪士尼世界的主要主题公园。 " , " output_ content ": " 正在搜索迪士尼官方来源。 " , " tool_ logs ": { " tool_name ": " web_search " , " i nput_arguments ": " Walt Disney World theme parks ", " outputs ": " 魔法王国、EPCOT、好莱坞影城和动物王国。 " , " execution_status ": " success " } }

此步骤涉及外部工具调用。原始工具交互在 tool_logs 中逐字保留。请注意，对于输出的每个步骤，都必须存在 output_content，并且只有存在工具调用时才会存在 tool_logs。

步骤 *3*：检索门票选项

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

```
{
  "step_id": 3,
  "agent_id": 1,
  "agent_name": "SearchAgent",
  "input_context": "Find ticket
      options for a three-day Disney
      World visit.",
  "output_content": "Searching ticket
      types and pricing.",
  "tool_logs": {
    "tool_name": "web_search",
    "input_arguments": "Disney World
        three-day ticket options",
    "outputs": "Standard tickets
        allow one park per day; park-
        hopper tickets allow multiple
         parks per day.",
    "execution_status": "success"
  }
}
```


Here, the same agent performs another retrieval step, producing additional evidence required for planning.

Step 4: Drafting an Initial Visit Plan


```
{
  "step_id": 4,
  "agent_id": 0,
  "agent_name": "PlannerAgent",
  "input_context": "Theme parks and
      ticket options have been
      retrieved.",
  "output_content": "Draft plan: Day
      1 Magic Kingdom, Day 2 EPCOT,
      Day 3 Hollywood Studios."
}
```


The planner synthesizes retrieved information into a tentative plan.

Step 5: Feasibility Verification


```
{
  "step_id": 5,
  "agent_id": 2,
  "agent_name": "VerifierAgent",
  "input_context": "Verify whether
      the proposed three-day plan is
      feasible for a family.",
  "output_content": "The plan is
      feasible. Park-hopper tickets
      are optional but not required."
}
```


The verifier checks internal consistency and practical feasibility.

Step 6: Final Recommendation


```
{
  "step_id": 6,
  "agent_id": 0,
  "agent_name": "PlannerAgent",
  "input_context": "Verification
      completed.",
  "output_content": "Final
      recommendation: Visit Magic
      Kingdom, EPCOT, and Hollywood
      Studios on separate days with
      standard three-day tickets."
}
```


This final step produces the consolidated answer returned to the user.

This example demonstrates how TraceElephant records: (1) rich trace metadata, (2) fine-grained step-level inputs and outputs, and (3) raw tool interactions, all in a unified JSON format.

A.5 Failure Attribution Annotation

This section describes the human annotation process used to obtain failure attribution labels, i.e., (1) the agent primarily responsible for the failure, and (2) the execution step where the failure originates.

Annotators. The annotation is conducted by three human annotators with at least one year of experience in developing and debugging MASs. The annotators are familiar with agent pipelines, tool-mediated execution, and trace-based analysis.

Annotation principle. The annotation process is conducted in three rounds. In the first round, failed execution traces from all multi-agent systems are evenly distributed among the three annotators. Each annotator independently inspects the complete execution trace, including agent actions and tool interactions. For each trace, the annotator identifies the failure-responsible agent and decisive failure step. Annotators additionally indicate whether they are confident or uncertain about each annotation.

In the second round, all traces marked as uncertain by at least one annotator are jointly reviewed. Annotators collaboratively examine the execution timeline, discuss alternative interpretations of the trace, and reconcile differences through discussion. Consensus is reached through mutual agreement rather than majority voting, and annotations are finalized only when all annotators agree on the attribution outcome.

{ " step_id ": 3, " agent_id ": 1, " agent_name ": " SearchAgent " , " input_context ": " 查找为期三天的迪士尼世界门票选项。", " output_content ": " 正在搜索门票类型和价格。" , " tool_logs ": { " tool_name ": " web_search " , " input_arguments ": " 迪士尼世界三日门票选项 " , " outputs ": " 标准门票允许每天游览一个园区；乐园通票允许每天游览多个园区。" , " execution_status ": " 成功 " } }

在这里，同一智能体执行另一次检索步骤，生成规划所需的额外证据。

步骤 *4*：起草初步游览计划

{ " step_id ": 4, " agent_id ": 0, " agent_name ": " PlannerAgent ", " input_context ": " The me parks and ticket options have been retrieved .", " output_content ": " Draft plan : Day 1 魔法王国 , Day 2 EPCOT , Day 3 好莱坞影城 ." }

规划器将检索到的信息综合成一份初步计划。

步骤 *5*：可行性验证

{ " 步骤_id ": 5, " 智能体_id ": 2, " 智能体_name ": " VerifierAgent ", " 输入_context ": " 验证所提出的三天计划对家庭而言是否可行 。", " 输出_content ": " 该计划是可行的 。公园多日通行票是可选的，但并非必需 。" }

验证器检查内部一致性和实际可行性。

步骤 *6*：最终建议

{ " 步骤_id ": 6, " 智能体_id ": 0, " 智能体_名称 ": " PlannerAgent ", " 输入 _ 上下文 ": " 验证已完成。", " 输出_内容 ": " 最终建议：在单独的日期参观魔法王国、EPCOT和好莱坞影城，并购买标准的三日通票。" }

此最终步骤生成返回给用户的合并答案。

该示例展示了 TraceElephant 如何记录：(1) 丰富的轨迹元数据，(2) 细粒度的步骤级输入和输出，以及 (3) 原始工具交互，全部采用统一的 JSON 格式。

A.5 失败归因标注

本节描述了用于获取故障归因标签的人工标注过程，即：(1) 对故障负主要责任的智能体，以及 (2) 故障起源所在的执行步骤。

标注员。标注工作由三名具有至少一年多智能体系统开发与调试经验的人工标注员完成。这些标注员熟悉智能体管道、工具介导的执行以及基于轨迹的分析。

标注原则。标注过程分三轮进行。在第一轮中，来自所有多智能体系统的失败执行轨迹被均匀地分配给三名标注员。每位标注员独立检查完整执行轨迹，包括智能体动作和工具交互。对于每条轨迹，标注员识别故障责任代理和决定性故障步骤。标注员还须表明他们对每条标注是有信心还是不确定。

在第二轮中，所有被至少一位标注者标记为不确定的执行轨迹将被共同审查。标注者协同查看执行时间线，讨论该轨迹的替代解释，并通过讨论弥合分歧。共识通过相互达成致来确立，而非通过多数表决；只有当所有标注者就归因结果达成一致时，标注才会被最终确定。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

In the final round, each annotator reviews a subset of annotations produced by the other annotators to assess consistency with the shared annotation standards. When inconsistencies or ambiguities are identified, the corresponding traces are revisited and re-annotated through further discussion until a consistent labeling standard is achieved across. This multi-round procedure follows established practices for aggregating expert judgments to approximate reliable ground truth (Krippendorff, 2018). We compute Krippendorff's alpha on the first-round independent annotations, obtaining 0.72 for agent-level labels and 0.64 for step-level labels. After the consensus process, all finalized labels reflect unanimous agreement among the annotators.

A.6 Evaluation Design

A.6.1 Evaluation Metrics.

Given an execution trace $\mathcal{O}(\mathcal{T})$, the failure attribution method is tasked with predicting the failure-responsible agent $\hat{a}$ and the decisive failure step $\hat{t}$. Predictions are evaluated against the ground-truth labels (a^*, t^*) annotated in our benchmark. Let $\mathcal{D}$ denote the set of failed execution traces in the benchmark. We report ***agent-level accuracy***, defined as the proportion of traces for which the predicted agent matches the ground truth: $\text{Acc}_{\text{agent}} = \frac{1}{|\mathcal{D}|}\sum_{(\mathcal{T},a^*,t^*)\in\mathcal{D}} \mathbb{I}[\hat{a} = a^*]$. Similarly, we report ***step-level accuracy***, defined as: $\text{Acc}_{\text{step}} = \frac{1}{|\mathcal{D}|}\sum_{(\mathcal{T},a^*,t^*)\in\mathcal{D}} \mathbb{I}[\hat{t} = t^*]$. Following prior work, we additionally report ***step-level accuracy with tolerance***, where a prediction is considered correct if $|\hat{t} - t^*| \leq \delta$, with δ fixed across all experiments. All results are averaged over 3 independent runs.

A.6.2 Observability Configurations.

We evaluate two categories of observability configurations. For ***Static*** configuration, we provide the overall trace information as demonstrated in Section 3.3 for failure attribution. For ***Dynamic*** configuration, besides the static information, we also provide a replayable execution environment for each trace. This setting reflects an interactive debugging workflow where static inspection is complemented by execution-based validation.

We additionally utilize ***Static w/o metadata***, ***Static w/o input***, and ***Static w/o metadata&input*** to further evaluate the necessity of different types of information for failure attribution.

A.6.3 Failure Attribution Techniques.

For above mentioned static configuration, we evaluate three commonly-used LLM-based failure attribution techniques (Zhang et al., 2025c,b; Ge et al., 2025) and two popular agent-based techniques (Yang et al., 2024). The first three techniques present the failure trace to the LLM, and let the LLM identify the responsible agent and decisive step. They differ in how the trace is provided. ***All-at-once*** provides the full trace in a single context window and asks the LLM to output the failure-responsible agent and decisive failure step. ***Step-by-step*** reveals the trace incrementally and asks the LLM to decide at each step whether it contains the failure origin, terminating when a step is selected. ***Binary search*** repeatedly asks the LLM to localize the failure to a sub-range of steps, halving the search space until a single step is identified.

For the first agent-based technique (short for ***Static Agentic***), we adopt mini-SWE-agent, which can navigate the trace information, retrieve related fields as needed, and make a conclusion gradually. Specifically, we modify the codebase of *mini-SWE-agent* by adding three tools: (1) inspecting the outputs of all executed steps; (2) inspecting the input and output of a specific step; and (3) submitting the final answer. These extensions enable the LLM agent to flexibly and dynamically switch between global information and fine-grained local details during reasoning.

For ***Dynamic Agentic***, the method first proposes candidate failure attributions derived from the Static Agentic technique, including candidate steps and agents, and then re-runs the system from the corresponding execution point to issue counterfactual checks. Specifically, we first leverage the Static Agentic method to infer up to n ($n \leq 3$) candidate *mistake agent/step/reason* triples, together with their corresponding *expected oracle*. The expected oracle represents the anticipated output of the identified step if the specified mistake reason had not occurred. It is inferred by the Static Agentic model as part of the candidate attribution hypothesis, rather than provided by human annotators or derived from the ground-truth labels. We then replay the task trajectory, and when execution reaches the identified mistake step, we intervene by modifying the input request of that step through an LLM API middleware. This intervention guides the agent to avoid the error induced by the mistake

在最后一轮中，每位标注员审查其他标注员生成的一小部分标注，以评估其与共享标注标准的一致性。当发现不一致或模糊之处时，将重新访问相应的执行轨迹，并通过进一步讨论重新标注，直至在所有标注员之间达成一致的标签标准。这一多轮程序遵循了聚合专家判断以近似可靠真值的既定做法（Krippendorff, 2018）。我们计算了第一轮独立标注的克里彭多夫阿尔法系数，智能体级标签的系数为0.72，步骤级标签的系数为0.64。经过共识过程后，所有最终标签均反映了标注员之间的一致同意。

A.6 评估设计

A.6.1 评估指标。

给定一个执行轨迹 $\mathcal{O}(\mathcal{T})$，故障归因方法需要预测故障责任代理 $\hat{a}$和关键故障步骤 t。预测结果将与在我们基准测试中标注的真实标签 (a^*, t^*) 进行对比评估。设 $\mathcal{D}$ 表示基准测试中的失败执行轨迹集合。我们报告智能体级准确率，定义为预测的代理与真值匹配的轨迹所占的比例：$\text{Acc}_{\text{agent}} = \frac{1}{|\mathcal{D}|}\sum_{(\mathcal{T},a^*,t^*)\in\mathcal{D}} \mathbb{I}[\hat{a} = a^*]$. 类似地，我们报告步骤级准确率，定义为：$\text{Acc}_{\text{step}} = \frac{1}{|\mathcal{D}|}\sum_{(\mathcal{T},a^*,t^*)\in\mathcal{D}} \mathbb{I}[\hat{t} = t^*]$. 遵循先前工作，我们还额外报告带有容差的步骤级准确率，其中如果 $|t - t^*| \leq \delta$，则该预测被视为正确，且 δ 在所有实验中固定不变。所有结果均为 3 次独立运行的平均值。

A.6.2 可观测性配置。

我们评估了两类可观测性配置。对于静态配置，我们提供了如第3.3节所示的整体轨迹信息，用于故障归因。对于动态配置，除了静态信息外，我们还为每个轨迹提供了一个可重放执行环境。该设置反映了一种交互式调试工作流，其中静态检查与基于执行的验证相互补充。

此外，我们还使用静态配置 w/o 元数据、静态配置 w/o 输入，以及静态配置 w/o 元数据&输入，以进一步评估不同类型信息对故障归因的必要性。

A.6.3 失败归因技术。

对于上述静态配置，我们评估了三种常用的基于大型语言模型的故障归因技术（Zhang et al., 2025c,b; Ge et al., 2025）和两种流行的基于智能体的技术（Yang et al., 2024）。前三种技术将故障轨迹呈现给大型语言模型，并要求其识别责任代理和决定性步骤。它们的区别在于轨迹的提供方式。全量输入将完整轨迹置于单一上下文窗口中，并要求大型语言模型输出故障责任代理和决定性故障步骤。逐步输入渐进式地展示轨迹，并要求大型语言模型在每一步判断其是否包含故障起源，直到选定某个步骤为止。二分查找反复要求大型语言模型将故障定位到步骤的一个子区间内，将搜索空间减半，直到识别出单个步骤。

对于第一种基于智能体的技术（简称为静态智能体），我们采用了mini-SWE智能体，它能够浏览轨迹信息，按需检索相关字段，并逐步得出结论。具体而言，我们通过添加三个工具修改了mini-SWE智能体的代码库：（1）检查所有已执行步骤的输出；（2）检查特定步骤的输入和输出；以及（3）提交最终答案。这些扩展使大语言模型智能体能够在推理过程中灵活、动态地在全局信息和细粒度本地细节之间切换。

对于动态智能体，该方法首先基于静态智能体技术提出候选故障归因，包括候选步骤和智能体，随后从相应的执行点重新运行系统以进行反事实检验。具体而言，我们首先利用静态智能体方法推断出最多 n （$n \leq 3$）个候选错误智能体/步骤/原因三元组，以及与之对应的预期神谕。预期神谕表示如果指定的错误原因未发生，被识别步骤的预期输出。该预期神谕由静态智能体模型作为候选归因假设的一部分进行推断，而非由人工标注者提供或从真实标签中推导。随后，我们重放任务轨迹，当执行到达被识别的错误步骤时，我们通过 LLM API 中间件修改该步骤的输入请求进行干预。该干预引导智能体避免由错误引发的行为。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

reason. We subsequently observe whether the next $k = 3$ steps satisfy the expected oracle and whether the previously observed failure modes reoccur or the agent deviates from the task objective.

We do not execute the entire task to completion; instead, we restrict our observation to $k = 3$ steps. This design choice is motivated by our empirical finding that many task failures stem from systemic design issues, where even corrective interventions may not guarantee success over long horizons. By focusing on a limited window of $k = 3$ steps, we assess local behavioral improvements within a bounded range, which provides a more reliable basis for diagnosis and judgment. Thus, Dynamic Agentic validates local responsibility for a candidate step, rather than attempting to repair the entire system or prove global causal sufficiency. We choose $k = 3$ because shorter windows often fail to expose meaningful behavioral changes, while longer windows introduce increasing variance from downstream stochasticity, tool noise, and unrelated planning effects. The decoding temperature is fixed at 0.3 to preserve local trajectory stability while still allowing limited behavioral adaptation after the intervention.

Note that there are other techniques for failure attribution in MASs (i.e., ECHO (Banerjee et al., 2025), AgenTracer (Zhang et al., 2025a), Graph-Tracer (Zhang et al., 2025b), and FAMAS (Ge et al., 2025)). However, they don't release runnable implementations or source code. We attempt to reproduce them, but couldn't obtain performance consistent with the numbers reported in original works. Therefore, we don't include them in evaluation.

A.6.4 Application Scenarios.

We evaluate attribution under two settings that simulate realistic debugging demands. In the ***With ground truth*** scenario, attribution methods are provided with task-level reference information, which is commonly available in development or diagnostic scenarios. Specifically, for GAIA and AssistantBench, we use the official task ground-truth answers as references. For SWE-Bench-Verified, where explicit reference answers are not available, we instead treat the observed test outcomes of the evaluation instances (i.e., pass/fail signals from the test suite) as ground-truth supervision. In contrast, in the ***Without ground truth*** scenario, the attribution method relies solely on the execution trace (and optional replay information), reflecting practical debugging scenarios where no definitive reference outcome is accessible.

A.6.5 LLMs Under Evaluation.

We evaluate failure attribution performance across a set of representative LLMs, covering both closed-source and open-source models. We use GPT-4o (Hurst et al., 2024) as the default LLM in the main experiments. We additionally compare against GPT-4o-mini (OpenAI, 2024), Claude-4.5-Sonnet (Anthropic, 2025) (closed-source) and Qwen3 (Yang et al., 2025), GPT-OSS-20B (Agarwal et al., 2025), DeepSeek-R1 (Guo et al., 2025) (open-source), to assess whether the findings are consistent across model families and capability levels.

A.7 Analysis on Who&When Benchmark

We report the distribution of failure agent and failure step in Who&When Benchmark in Figure 8 and 9. These trends align closely with those observed in TraceElephant (see Figure 4 and 6).

The slight difference lies in the average number of LLM invocations: in Who&When, Captain-Agent and Magentic-One respectively have an average of 9.6 and 28.8 calls, while these figures for TraceElephant are 20.5 and 29.3.

It is also worth noting that the step count for Magentic-One appears significantly higher (almost 80 steps) in Who&When as shown in Figure 9; this discrepancy stems from differences in step-counting strategies between the two benchmarks. For example, CaptainAgent omits the scheduling process among speaking agents, and ComputerTerminal executes code from the previous step directly via the command line without invoking an LLM. In Magentic-One, the process of reasoning about and selecting the next speaking agent is decomposed into multiple steps. We reconstructed the actual number of LLM invocations in the Who&When dataset: Algorithm-Generated (Captain-Agent) averages 9.6 invocations, Hand-Crafted (Magentic-One) averages 28.8, and the overall average is 15.7.

A.8 Privacy Considerations

TraceElephant is constructed using execution traces that include step-level inputs and outputs of agents. While such information is essential for fine-grained failure attribution, it may contain sensitive content, depending on the task domain and system configuration. For example, execution traces may include user-provided instructions, intermediate reasoning

推理。随后，我们观察接下来的 $k = 3$ 个步骤是否满足预期的预言机，以及先前观察到的失效模式是否会再次出现，或者智能体是否会偏离任务目标。

我们不会执行整个任务直至完成，而是将我们的观察限制在 $k = 3$ 步骤内。这一设计选择源于我们的经验发现，即许多任务失败源于系统性设计问题，在这种情况下，即使在长期范围内进行纠正性干预，也可能无法保证成功。通过聚焦于 $k = 3$ 步骤的有限窗口，我们评估在有限范围内的局部行为改进，这为诊断和判断提供了更可靠的基础。因此，动态智能体验证了候选步骤的局部责任，而不是试图修复整个系统或证明全局因果充分性。我们选择 $k = 3$，因为较短的窗口往往无法揭示有意义的行为变化，而较长的窗口则会引入来自下游随机性、工具噪声以及无关规划效应的逐渐增加的方差。解码温度固定在 0.3，以在干预后仍允许有限的行为适应的同时，保持局部轨迹稳定性。

还存在其他用于多智能体系统故障归因的技术（即ECHO（Banerjee等人，2025）、AgenTracer（Zhang等人，2025a）、Graph-Tracer（Zhang等人，2025b）和FAMAS（Ge等人，2025））。然而，它们未发布可运行的实现或源代码。我们尝试复现这些方法，但未能获得与原始论文中报告的性能一致的结果。因此，我们未将其纳入评估。

A.6.4 应用场景。

我们在模拟真实调试需求的两种设置下评估归因。在“含真值”场景中，归因方法可获得任务级的参考信息，这类信息在开发或诊断场景中通常可用。具体而言，对于GAIA和AssistantBench，我们使用官方任务的真值答案作为参考。对于SWE-Bench-Verified，由于没有明确的参考答案，我们将评估实例（即测试套件的通过/失败信号）所观察到的测试结果作为真值监督。相比之下，在“不含真值”场景中，归因方法仅依赖执行轨迹（以及可选的重放信息），这反映了在实际调试场景中，无法访问明确的参考结果。

A.6.5 待评估的大语言模型

我们在一组具有代表性的大型语言模型（LLM）上评估了故障归因性能，涵盖闭源和开源模型。在主实验中，我们使用GPT-4o（Hurst等人，2024）作为默认的大型语言模型。此外，我们还与GPT-4o-mini（OpenAI，2024）、Claude-4.5-Sonnet（Anthropic，2025年）（闭源）以及Qwen3（Yang等人，2025年）、GPT-OSS-20B（Agarwal等人，2025年）、DeepSeek-R1（Guo等人，2025年）（开源）进行了比较，以评估研究结论在不同模型系列和能力级别之间是否具有一致性。

A.7 Who&When基准分析

我们在图8和图9中报告了Who&When基准中故障代理和故障步骤的分布。这些趋势与在TraceElephant中观察到的趋势高度一致（见图4和图6）。

细微的差别在于大语言模型调用的平均次数：在Who&When中，Captain-Agent和Magentic-One分别平均有9.6和28.8次调用，而在TraceElephant中，这些数值分别为20.5和29.3。

值得注意的是，如图9所示，Magentic-One在Who&When中的步骤计数似乎显著较高（近80步）；这一差异源于两个基准在步骤计数策略上的不同。例如，CaptainAgent会省略发言智能体中的调度过程，而ComputerTerminal会直接通过命令行执行上一步的代码，而不调用大型语言模型。在Magentic-One中，推理并选择下一个发言智能体的过程被分解为多个步骤。我们重建了Who&When数据集中大语言模型调用的实际次数：算法生成（Captain-Agent）平均有9.6次调用，手工构建（Magentic-One）平均有28.8次，总体平均为15.7次。

A.8 隐私考量

TraceElephant基于包含智能体步骤级输入和输出的执行轨迹构建。尽管此类信息对于细粒度故障归因至关重要，但它可能包含敏感内容，具体取决于任务领域和系统配置。例如，执行轨迹可能包含用户提供的指令、中间推理

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

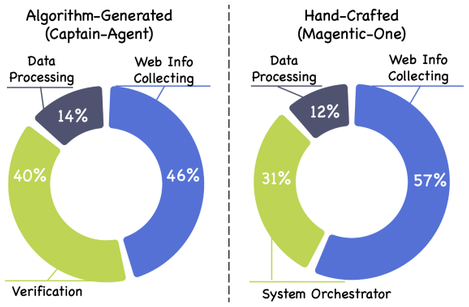


Figure 8: Distribution of failure agent in Who&When.

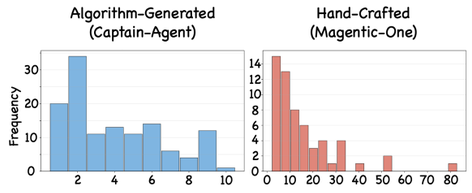


Figure 9: Distribution of failure step in Who&When.

artifacts, or tool invocation details that could expose proprietary logic or private information.

To reduce potential privacy risks, we adopt several conservative design choices during benchmark construction. First, all traces are collected from controlled task executions over public benchmark sources, rather than from real user interactions. Tasks are predefined and do not involve personal user data. Second, for web-based executions, we use an isolated Playwright-controlled Chromium instance without personal browsing profiles, cookies, or user-specific history. Third, we apply automatic filtering to remove local file paths and potential personal identifiers from collected traces, followed by manual inspection of each released trace. Fourth, when releasing the benchmark, sensitive fields can be selectively anonymized or transformed, for example by replacing raw inputs or outputs with hashed or redacted variants when appropriate.

A.9 Human Annotation Protocol and Ethics Statement

We employed three expert annotators with prior experience in developing and debugging LLM-based multi-agent systems to provide failure attribution labels for the benchmark. Each annotator was instructed to identify (i) the failure-responsible agent and (ii) the earliest execution step at which task failure becomes inevitable, based on the complete execution trace, including agent inputs, outputs, and tool interactions.

The annotation guidelines emphasized consistency, trace-based reasoning, and adherence to the formal definition of step-level and agent-level failure attribution described in Appendix 2. Annotators conducted their analysis independently in the first round, followed by joint discussion and reconciliation for cases marked as uncertain, until consensus was reached.

All annotators participated voluntarily and were fully informed of the purpose of the study and the intended use of the annotated data for research and benchmarking purposes. The annotation task involved only synthetic tasks or publicly available benchmark data (GAIA, AssistantBench, and SWE-Bench) and did not include any personal, sensitive, or user-generated data.

No external recruitment or crowdsourcing platforms were used, and no demographic data were collected. As the study involved only technical annotation of non-sensitive data by informed research participants, it did not require approval from an institutional ethics review board.

A.10 Artifact License and Usage Terms

TraceElephant including execution traces, annotations, and associated metadata, is released for research purposes only. All benchmark artifacts will be made publicly available upon publication through an open-access repository.

The execution traces and annotations are distributed under the Creative Commons Attribution 4.0 International (CC BY 4.0) license, which permits use, distribution, and adaptation for research purposes with appropriate attribution. Before release, the artifacts are filtered and manually reviewed to avoid including personal, sensitive, or proprietary information.

For third-party components referenced in the benchmark, including task sources such as GAIA, AssistantBench, and SWE-Bench, we follow their original licenses and terms of use. Users of TraceElephant are responsible for complying with the licenses of these upstream resources when using or redistributing the benchmark.

The benchmark code, including trace collection scripts and evaluation utilities, will be released under a permissive open-source license, and detailed licensing information will be provided in the accompanying repository.

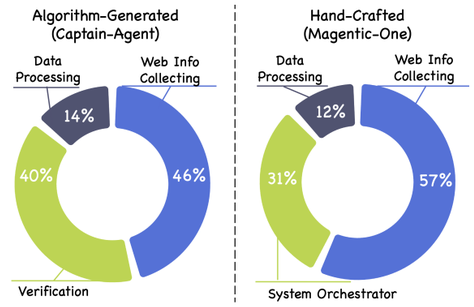


图 8：Who&When 中故障代理的分布。

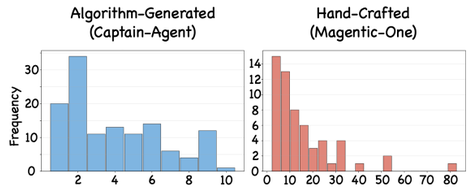


图9：Who&When 中故障步骤的分布。

产物或工具调用细节，这些信息可能会泄露专有逻辑或私有信息。

为降低潜在的隐私风险，我们在基准构建过程中采用了若干保守的设计选择。首先，所有执行轨迹均收集自基于公开基准源的控制任务执行，而非来自真实用户交互。这些任务是预先定义的，不涉及个人用户数据。其次，对于基于网络的执行，我们使用一个隔离的、由Playwright控制的Chromium实例，该实例不包含个人浏览配置、Cookie或特定用户历史。第三，我们应用自动过滤，从收集到的执行轨迹中移除本地文件路径和潜在的个人标识符，随后对每个发布的执行轨迹进行人工检查。第四，在发布基准测试时，可以选择性地对敏感字段进行匿名化或转换，例如，在适当的情况下，用哈希或删减变体替换原始的输入或输出。

A.9 人工标注协议与伦理声明

我们聘请了三位具有开发和调试基于大型语言模型的多智能体系统经验的专家标注员，为该基准测试提供故障归因标签。每位标注员均被要求识别（i）故障责任代理以及（ii）任务失败变得不可避免，基于完整的执行轨迹，包括智能体输入、输出以及工具交互。

标注指南强调了一致性、基于轨迹的推理，以及遵循附录2中所述的步骤级和智能体级故障归因的正式定义。标注员在第一轮中独立进行分析，随后针对被标记为不确定的案例进行联合讨论和对齐，直至达成共识。

所有标注员均自愿参与，并充分了解研究目的以及标注数据用于研究和基准测试的意图。标注任务仅涉及合成任务或公开可用的基准测试数据（GAIA、AssistantBench 和 SWE-Bench），不包含任何个人、敏感或用户生成的数据。

未使用任何外部招募或众包平台，也未收集人口统计数据。由于本研究仅涉及由知情研究参与者对非敏感数据进行技术性标注，因此无需获得机构伦理审查委员会的批准。

A.10 工件许可证和使用条款

TraceElephant 包含执行轨迹、注释及相关元数据，仅用于研究目的。所有基准工件将在论文发表后通过开放获取存储库向公众公开。

执行轨迹和标注在知识共享署名4.0国际（CC BY 4.0）许可证下分发，该许可证允许在适当归因的情况下，出于研究目的进行使用、分发和改编。在发布之前，产物经过筛选并手动审查，以避免包含个人、敏感或专有信息。

对于基准测试中引用的第三方组件，包括 GAIA 、 AssistantBench 和 SWE-Bench 等任务来源，我们遵循其原始许可证和使用条款。TraceElephant 的用户在使用或再分发该基准测试时，有责任遵守这些上游资源的许可证。

基准测试代码，包括轨迹收集脚本和评估工具，将以宽松开源许可证发布，并将在配套代码库中提供详细的许可信息。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next